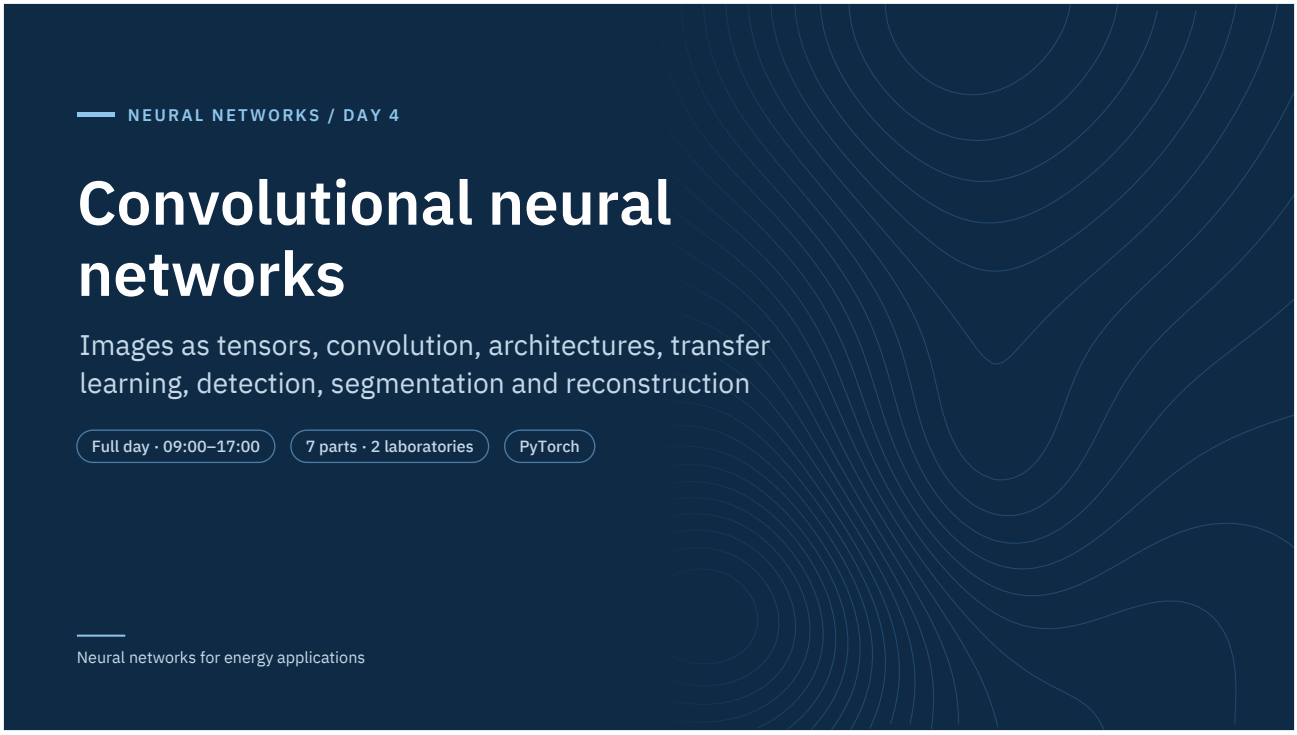
Convolutional neural networks

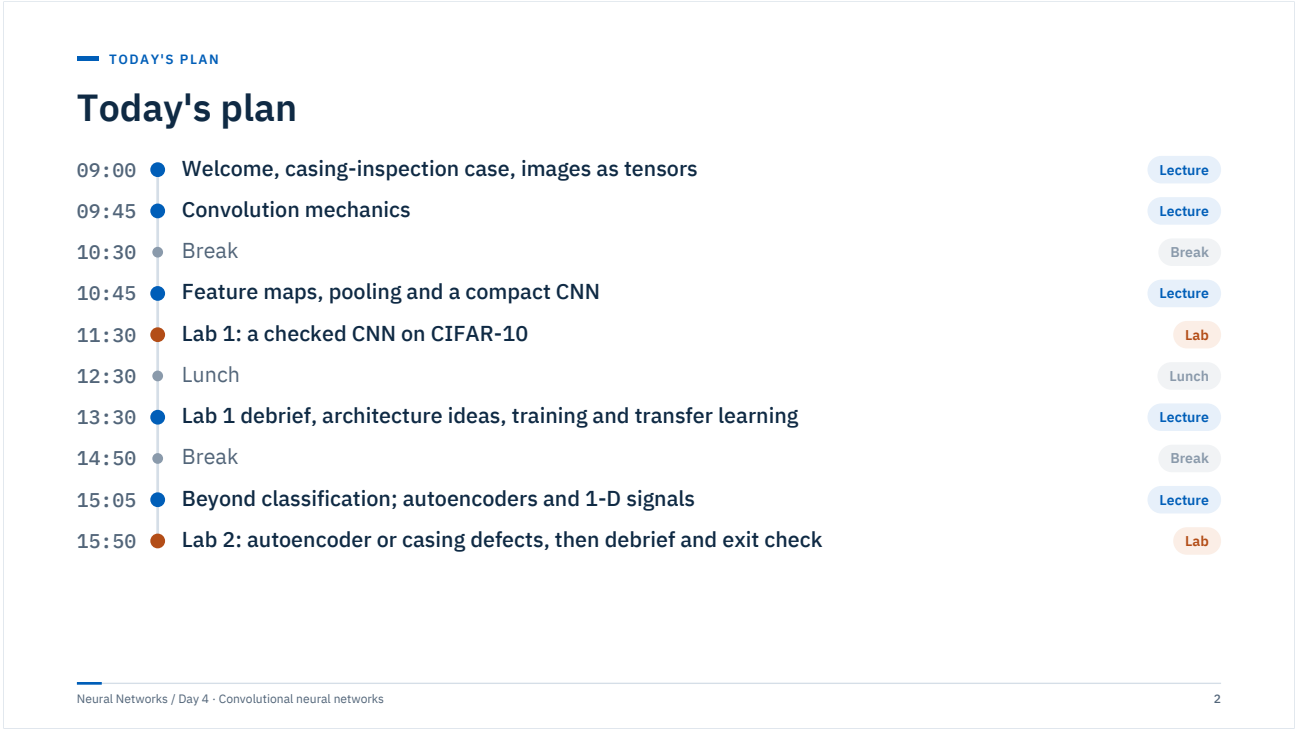


Day 4 builds on dense networks, gradients, loss functions, training loops and train/eval modes from Days 1 and 2. The day uses images and ordered signals to motivate two ideas, locality and shared weights, and writes out every tensor shape so learners can debug their own architecture changes.

Preparation before class: install torchvision; download CIFAR-10 (Lab 1) and MNIST (Lab 2A) into each notebook's ./data folder, or into one shared folder named by the COURSE_DATA environment variable, which the notebooks read; install h5py for Lab 2B, which reads the local HDF5 casing file. The teaching figures need no downloads: they use synthetic data, scikit-learn's bundled digits and its bundled sample photograph.

Say at the start that the energy examples are illustrations. No slide claims field performance, and every synthetic figure says so in its caption.

Today's plan



[2 min] The morning builds the mechanics: tensors, convolution arithmetic, pooling and a compact CNN, ending with Lab 1 just before lunch so that training runs can continue while people eat if needed. The Lab 1 debrief opens the afternoon (13:30), followed by architecture ideas (13:40) and training and transfer learning (14:10). After the break, beyond classification runs 15:05–15:30 and autoencoders and 1-D signals 15:30–15:50. Lab 2 runs 15:50–16:45; the debrief, recap and exit check take the last 15 minutes.

The afternoon moves from "how does it work" to "how do we use it responsibly": architecture ideas, augmentation, class imbalance, grouped splits and transfer learning, then other output types (boxes, masks, reconstructions, sequences). Lab 2 offers two paths: a denoising autoencoder (MNIST) or casing-defect classification on the local ultrasonic HDF5 file.

If the downloads failed before class, start the CIFAR-10 download (about 170 MB) at the beginning of Lab 1 while pairs write the shape check, and lower N_TRAIN to 10,000 to win back the time; Lab 2B needs no download. Keep the debrief and exit check that start at 16:45 even if a lab overruns: the exit check is the evidence that the day's outcomes were met.

What you will be able to do

LEARNING OUTCOMES

What you will be able to do

By the end of the day you will be able to:

1. Represent images and energy imaging data as tensors with the right shape and meaning.
2. Compute convolution outputs, output sizes, parameter counts and receptive fields by hand.
3. Build, shape-check and train a compact CNN in PyTorch.
4. Adapt a pretrained backbone with label-preserving augmentation and grouped evaluation.
5. Choose between classification, detection and segmentation, and test what a model uses.
6. Use convolutional autoencoders and 1-D convolutions without overstating what they show.

Neural Networks / Day 4 · Convolutional neural networks 3

[2 min] Read the outcomes as observable skills. Outcomes 1 and 2 are checked by the hand exercises in Parts 1 to 3; outcome 3 by Lab 1; outcome 4 by the augmentation and adaptation exercises and Lab 2B; outcome 5 by the IoU exercise and the shortcut discussion; outcome 6 by Lab 2A and the exit check.

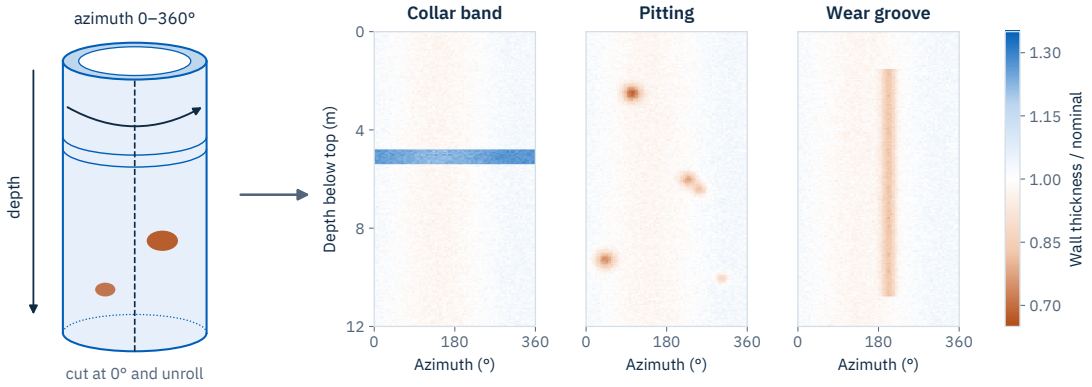
Ask the room which of these they have already done at work. Typical answers: people have fine-tuned a pretrained image model from a tutorial but have never computed an output size by hand or questioned a random patch split. Today's emphasis is on the second group of skills, because those are what make a CNN result defensible to an engineer or an auditor.

A casing inspection log is already an image

NEURAL NETWORKS / DAY 4

A casing inspection log is already an image

Ultrasonic imagers record wall thickness around the pipe at each depth: a depth × azimuth grid.



Synthetic thickness maps (seeds 4–6) with 0.05 m rows and 5° sectors, drawn to show the geometry; not tool data.

[3 min] Start with an energy image the room recognizes. An ultrasonic casing imager measures the steel wall at many azimuths around the pipe at each depth. Cut the cylinder at 0° and unroll it, and you have an image: rows are depth, columns are azimuth, and each value is a thickness.

Point to the three synthetic intervals. A collar shows as a horizontal band of different thickness, pitting as small local thin spots, and a wear groove as a long narrow thin stripe at one azimuth. Orange means thinner than nominal, blue thicker.

Ask: what makes a pit a pit? It is a local pattern, a small neighbourhood thinner than its surroundings, that can occur at any depth and any azimuth. That is exactly the assumption a convolutional network builds in, which is why the morning is about convolution.

These maps are synthetic and only show geometry. Lab 2B uses a public ultrasonic dataset whose labels and grouping we will inspect before trusting any score.

Decide what the model should return

PRACTICE

4 min Pairs

Decide what the model should return

For the casing maps on the previous slide:

1. Should the model return one label per image, per depth window, or per pixel?
2. Which neighbouring pixels would you inspect to judge whether one pixel is a pit?
3. Two windows from one well overlap by half. May one be in training and the other in test?

Neural Networks / Day 4 · Convolutional neural networks 5

[4 min in pairs, 2 min discussion] There is no single right answer to question 1, and that is the point.

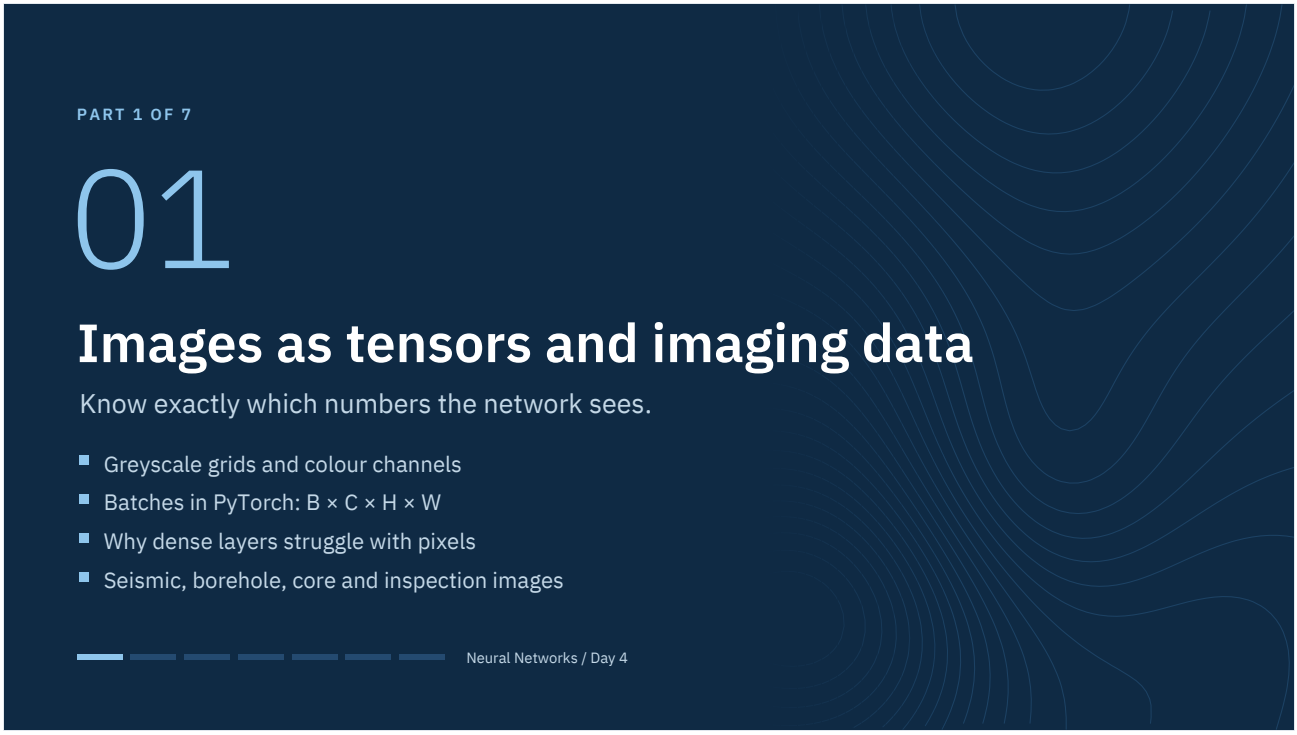
1. It depends on the decision. A screening tool may only need a flag per joint or per depth window. An engineer planning a repair needs the depth interval, azimuth and extent, which means boxes or per-pixel masks. This previews Part 6: classification, detection and segmentation differ in output, labels and metrics.

2. A small window around the pixel: its neighbours in depth and azimuth compared with the surrounding wall. Nobody needs the whole well to judge one pixel. This previews locality, receptive fields and why convolution uses small shared filters.

3. No. Overlapping windows share half their pixels, so a model can memorize one and be "tested" on its near-copy. Split by well, or at least by non-overlapping depth blocks with a gap. This previews grouped evaluation in Part 5 and Lab 2B.

Common error: answering question 1 with "whatever gives the highest accuracy". The output format is a design decision made before modelling.

Images as tensors and imaging data



[About 30 minutes for this part, including two short exercises.] The goal is that nobody leaves this part unsure what the four axes of an image batch mean, or why a dense layer on raw pixels is a poor starting point.

We start with a tiny greyscale digit, move to a colour photograph, write the PyTorch layout explicitly, count parameters, and then look at the energy imaging modalities that motivate the rest of the day. Keep the pace brisk for experienced learners, but make sure the tensor-layout slide lands: shape mistakes are the most common first bug in CNN code.

A greyscale image is a grid of numbers

IMAGES AS TENSORS AND IMAGING DATA

A greyscale image is a grid of numbers

- A scikit-learn digit is an 8 × 8 array of intensities from 0 to 16.
- Row and column indices carry meaning: neighbours are physically adjacent.
- Flattening to 64 numbers keeps the values but hides which ones are neighbours.

	0	1	2	3	4	5	6	7
0	0	0	7	15	13	1	0	0
1	0	8	13	6	15	4	0	0
2	0	2	1	13	13	0	0	0
3	0	0	2	15	11	1	0	0
4	0	0	0	1	12	12	1	0
5	0	0	0	0	1	10	8	0
6	0	0	8	4	5	14	9	0
7	0	0	7	13	13	9	0	0

rendered



8 × 8 array

64 intensities

values 0–16

x[1, 4] = 15

row 1, column 4

scikit-learn bundled digit (index 3); exact pixel values.

Neural Networks / Day 4 · Convolutional neural networks 7

[2 min] This is the same digit used later in the filter examples. Each cell is one pixel; the number is its intensity on scikit-learn's 0 to 16 scale, where 0 is background and 16 is full ink. The highlighted cell is row 1, column 4, with value 15.

The key point is the last bullet. A dense network sees a flattened vector of 64 numbers. It can learn anything in principle, but nothing in its structure says that pixel (1, 4) is next to pixel (1, 5). If you permuted the 64 pixels in the same way for every image, a dense network would train just as well; a convolutional network would not, because it relies on the grid.

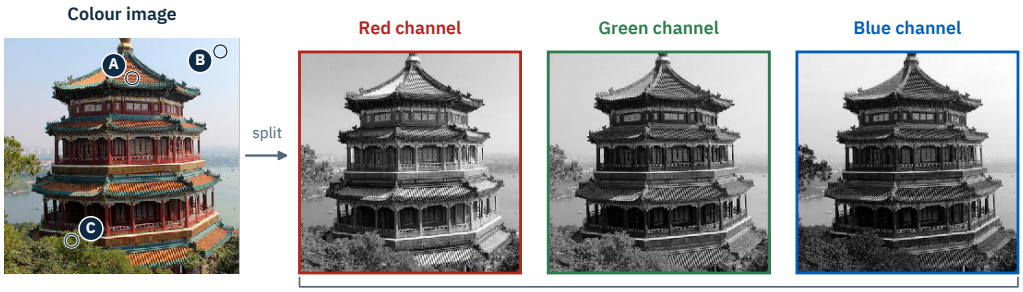
Ask: what does that permutation experiment tell you about when convolution is appropriate? Answer: only when the arrangement of values carries meaning, as in images, spectra, logs and time series.

A colour image stacks three channel arrays

IMAGES AS TENSORS AND IMAGING DATA

A colour image stacks three channel arrays

Each pixel is a vector of three numbers; each channel on its own is a greyscale image.



A roof: (R, G, B) = (209, 121, 81)
B sky: (R, G, B) = (216, 234, 254)
C foliage: (R, G, B) = (116, 125, 42)

NumPy / PIL: H × W × C = 360 × 360 × 3, integers 0 (dark) to 255 (bright)

PyTorch: C × H × W = 3 × 360 × 360, usually rescaled to floats in [0, 1]

Crop of scikit-learn's sample image china.jpg (CC BY 2.0, danielbuechele on Flickr); exact 8-bit values.

Neural Networks / Day 4 · Convolutional neural networks 8

[2 min] A colour photograph stores three numbers per pixel: red, green and blue intensities from 0 to 255. Split them and you get three greyscale images of the same size. The orange roof pixel A is (209, 121, 81): strong red, weak blue. The sky pixel B is bright in all three channels, bluest in blue. The foliage pixel C has little blue.

Point to the bottom lines. NumPy and image libraries such as PIL store a colour image as height × width × channels. PyTorch layers expect channels first, and usually floating-point values rescaled to 0–1. Many energy images have one channel (thickness, amplitude, resistivity) or many (multispectral drone imagery, several seismic attributes stacked as channels). The channel axis is simply "several measurements at the same location".

PyTorch expects batches shaped B × C × H × W

IMAGES AS TENSORS AND IMAGING DATA

PyTorch expects batches shaped B × C × H × W

```
import torch
from sklearn.datasets import load_sample_image

img = load_sample_image("china.jpg")        # NumPy array: H x W x C
print(img.shape, img.dtype)

x = torch.tensor(img).permute(2, 0, 1)      # C x H x W (a copy)
x = x.float().div(255).unsqueeze(0)         # B x C x H x W, values 0-1
print(x.shape, x.dtype)
```

OUTPUT

```
(427, 640, 3) uint8
torch.Size([1, 3, 427, 640]) torch.float32
```

Keras and many image libraries default to channels-last (B × H × W × C): check before you reshape.

[3 min] Walk through line by line. The image arrives as a 427 × 640 × 3 array of unsigned 8-bit integers. permute(2, 0, 1) reorders the axes to channels, height, width; it does not move any pixel values. The float conversion and division by 255 give values between 0 and 1, and unsqueeze(0) adds the batch axis, so a single image becomes a batch of one.

We use torch.tensor, which copies the data, because scikit-learn returns a read-only array: torch.from_numpy would share its memory and warn that the array is not writable.

Common errors: using reshape or view instead of permute, which silently scrambles pixels into the wrong channels; forgetting the batch axis, which gives a shape error in Conv2d; and feeding 0–255 integers to a model trained on normalized floats. Ask the room to state the shape of a batch of 64 greyscale 28 × 28 images: 64 × 1 × 28 × 28.

Locality reduces the parameter burden

PRACTICE

4 min · Individual

Locality reduces the parameter burden

An RGB image has shape 32 × 32 × 3.

1. How many weights connect every pixel value to 100 dense units?
2. How many weights define 16 filters of size 3 × 3 over the same input channels?
3. Which of the two counts changes if the image grows to 224 × 224?

[4 min individual, then reveal the next slide]

1. Dense: every one of the 32 × 32 × 3 = 3,072 input values connects to each of 100 units, so 3,072 × 100 = 307,200 weights, plus 100 biases.

2. Convolution: each filter spans all 3 input channels and a 3 × 3 window, so 3 × 3 × 3 = 27 weights per filter; 16 filters give 16 × 27 = 432 weights, plus 16 biases (448 parameters in total).

3. Only the dense count changes. At 224 × 224 × 3 = 150,528 inputs, 100 dense units need 15,052,800 weights. The convolution still has 432 weights, because the same filters are applied at every position.

Common error: forgetting that a convolution filter spans all input channels, giving 16 × 9 = 144 instead of 432. The computation and activation memory of the convolution do grow with the image; only its parameter count does not.

Dense weight counts grow with every pixel

DISCUSSION

Dense weight counts grow with every pixel

Weights (biases excluded) for 100 dense units versus 16 convolution filters of size 3 × 3:

307,200	15,052,800	432
dense layer, 32 × 32 × 3 input	dense layer, 224 × 224 × 3 input	16 conv filters, any image size

◆ Convolution saves weights by assuming local patterns that recur; compute still grows with the pixel count.

Neural Networks / Day 4 · Convolutional neural networks 11

[2 min] These are the answers to the exercise. Emphasize the difference between parameters and computation. The 432 convolution weights are reused at every output position: with padding that keeps the size, a 32 × 32 input needs 32 × 32 × 432 = 442,368 multiply-adds for this layer, and a 224 × 224 input needs 50,176 × 432 = 21,676,032. Parameters are fixed; compute scales with pixels.

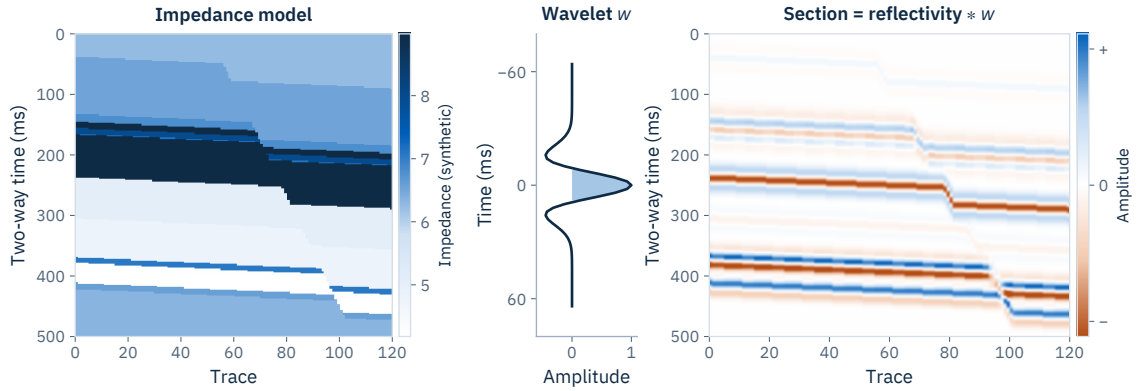
Fewer parameters is not a free lunch. It works because images really do contain local patterns that recur at different positions. If that assumption fails, for example with shuffled table columns, convolution throws away information a dense layer could have used.

Seismic sections are images made by convolution

IMAGES AS TENSORS AND IMAGING DATA

Seismic sections are images made by convolution

Each trace is reflectivity convolved with a wavelet; faults offset the events.



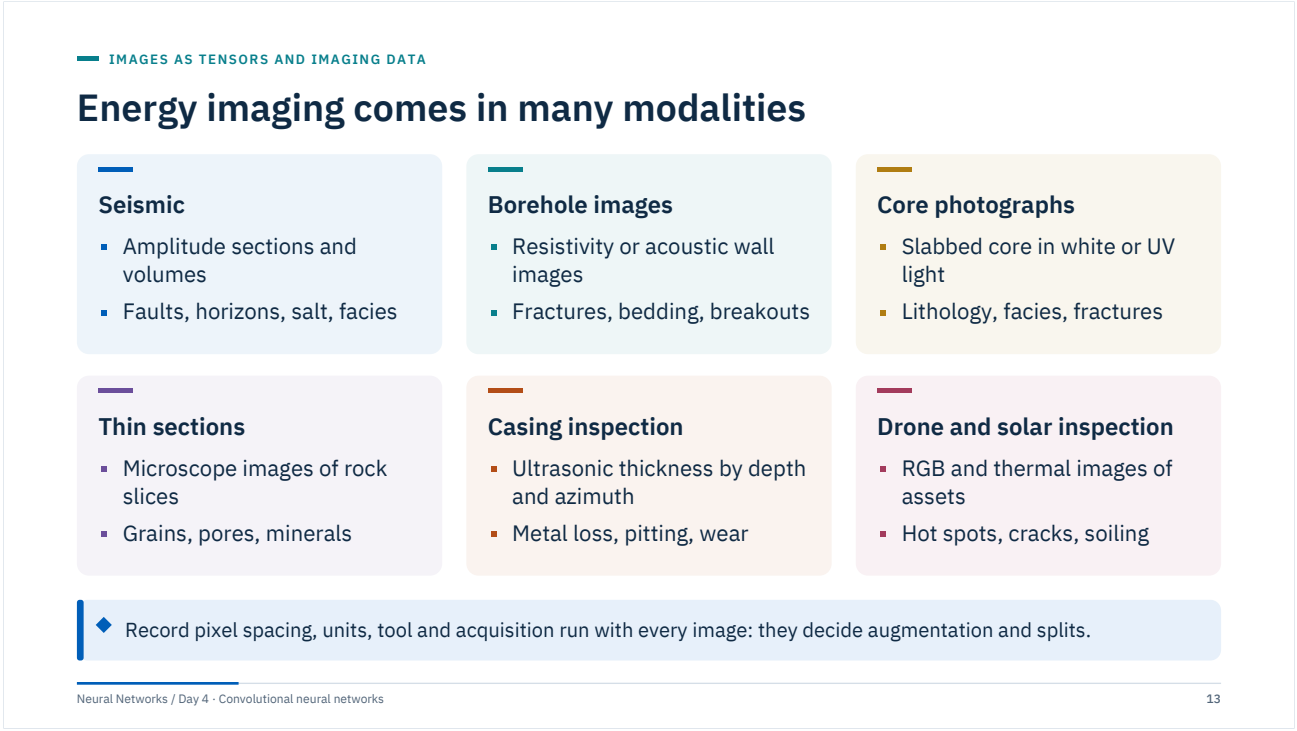
Synthetic impedance model with a normal fault (seed 7), 25 Hz Ricker wavelet, 2 ms sampling; not field data.

[3 min] Left: a layered acoustic-impedance model with a normal fault; the right-hand block has moved down by 14 samples (28 ms). Reflectivity at each interface is r = (Z2 − Z1)/(Z2 + Z1). Middle: a 25 Hz Ricker wavelet, $w(t) = (1 - 2\pi^2 f^2 t^2)\exp(-\pi^2 f^2 t^2)$. Right: each trace of reflectivity convolved with the wavelet gives the synthetic section.

Two lessons. First, a seismic section is an image whose vertical axis is two-way time, with a characteristic spacing (2 ms here) and blurring set by the wavelet. Second, the word convolution appears twice today with different roles: here it is the physical forward model of the recording; in a CNN it is a learned filter bank. A CNN kernel is not the wavelet, and learning it does not invert the physics.

The fault is visible as an offset of continuous events: a local pattern, which is why segmentation networks are used for fault picking. We return to this section in Part 6.

Energy imaging comes in many modalities



[3 min] Each card has the same structure: what the pixels are, then typical targets. Borehole images, like casing maps, are unwrapped cylinders with a depth axis and an azimuth axis. Core photographs and thin sections are ordinary RGB images with a physical scale. Drone inspection mixes RGB and thermal channels.

Ask the room which modalities they work with and what the pixel spacing is. Two practical points follow. First, spacing differs by axis: a borehole image may have fine depth sampling and fewer azimuth sectors, so square kernels do not see square physical areas. Second, acquisition metadata (tool, logging run, survey, vintage) is often what a model accidentally learns. Keep it with the data so that you can split by it and test for it.

We will not claim results for any modality; these are the settings where the day's methods are used.

Every tensor axis has a physical meaning

PRACTICE

4 min Pairs

Every tensor axis has a physical meaning

A batch holds 16 casing patches of 32 depth rows × 36 azimuth sectors of 10°, one thickness channel; each row spans 0.1 m.

1. Write the PyTorch shape of the batch.
2. What depth interval and azimuth range does one patch cover?
3. Which axis wraps around, and what does that mean at the patch edges?

Neural Networks / Day 4 · Convolutional neural networks 14

[4 min in pairs, 2 min discussion]

1. (16, 1, 32, 36): batch, channel, depth, azimuth. A common error is channels-last (16, 32, 36, 1) or dropping the channel axis.

2. Depth: 32 rows × 0.1 m = 3.2 m. Azimuth: 36 sectors × 10° = 360°, so the patch covers the whole circumference.

3. Azimuth wraps: sector 35 is physically adjacent to sector 0. If a convolution pads the azimuth edges with zeros, border outputs see an artificial "zero thickness" wall that does not exist, and a pit straddling 0° is cut in two. The fix is circular padding along azimuth only, shown in Part 2. Depth does not wrap: the next rows belong to a different depth, possibly a different joint.

Bridge: this is the first example where the physics of acquisition decides a network detail. It will not be the last.

Convolution mechanics



[About 45 minutes for this part.] This is the arithmetic core of the day. Learners should leave able to compute any Conv2d output shape and parameter count by hand; Lab 1 depends on it.

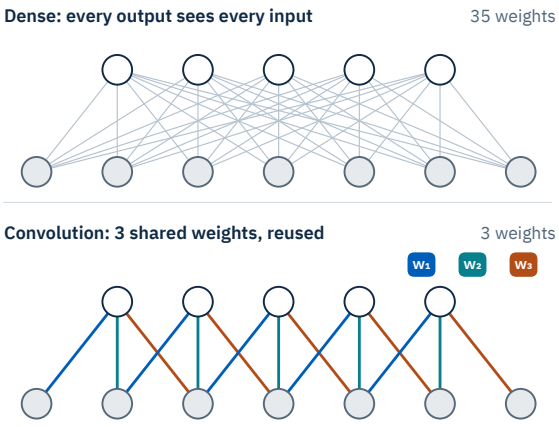
Pace: three slides on the idea of a shared filter (one worked by hand), then padding with the azimuth-wrap example, stride, dilation, the general output-size formula, a shape exercise, multi-channel convolution, the 1 × 1 case and a parameter-count exercise. Encourage learners to write the numbers down rather than just watch.

A filter applies the same rule across positions

CONVOLUTION MECHANICS

A filter applies the same rule across positions

- Each output combines a local neighbourhood of the input with learned weights.
- The same weights are reused at every position: **weight sharing**.
- A filter spans every input channel; each filter makes one output channel.
- Sharing encodes an assumption: a useful local pattern can recur anywhere.



Schematic: seven inputs, five outputs, kernel size 3, no padding.

Neural Networks / Day 4 · Convolutional neural networks 16

[3 min] Top: a dense layer from 7 inputs to 5 outputs has 35 separate weights. Bottom: a convolution with kernel size 3 connects each output only to three neighbouring inputs, and every output uses the same three weights, shown by colour. Three weights instead of 35, and the rule is identical at every position.

Most deep-learning libraries implement cross-correlation, sliding the kernel without flipping it, while keeping the conventional name convolution. Because the kernel is learned, the difference does not matter for training, but it matters if you compare with a textbook or with signal-processing code.

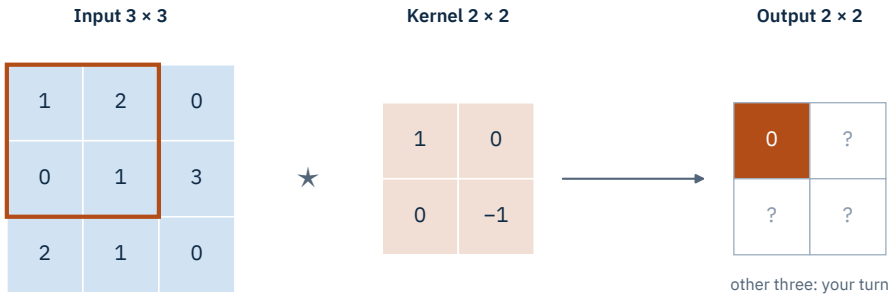
Locality is useful for images and signals, but not automatically for arbitrarily ordered tabular columns. Ask: which of your datasets have a meaningful neighbourhood structure? Logs over depth, sensor series over time and images do; a table of well attributes usually does not.

One output value is a weighted sum of a patch

CONVOLUTION MECHANICS

One output value is a weighted sum of a patch

Multiply the highlighted patch by the kernel element-wise and add: 1·1 + 2·0 + 0·0 + 1·(−1) = 0.



Exact cross-correlation (scipy.signal.correlate2d) with one 2 × 2 kernel, stride 1, no padding; three outputs left for the room.

Neural Networks / Day 4 · Convolutional neural networks 17

[3 min] Use the displayed values to calculate the first output with the class: the highlighted 2 × 2 patch is [[1, 2], [0, 1]]; multiplied element-wise by the kernel [[1, 0], [0, −1]] and summed, it gives 1 + 0 + 0 − 1 = 0. The star between input and kernel is the usual symbol for cross-correlation, the operation that frameworks call convolution.

Then ask everyone to compute the other three outputs before revealing the next slide. Learned kernels can have negative values; this one responds to a difference between a pixel and its lower-right neighbour.

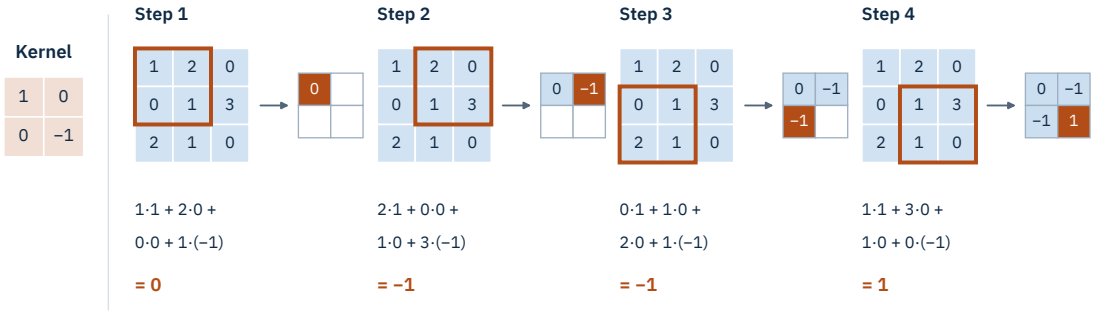
Note that the output (2 × 2) is smaller than the input (3 × 3) because no padding is used: a 2-wide window fits in two positions along each 3-wide axis. Distinguish this numerical operation from the decorative architecture pictures that learners will see in papers; every block in those pictures is this arithmetic repeated many times.

The kernel slides to fill the output map

CONVOLUTION MECHANICS

The kernel slides to fill the output map

Frameworks compute cross-correlation: the kernel slides without being flipped.



Exact arithmetic on the same numbers as the previous slide; stride 1, no padding.

Neural Networks / Day 4 · Convolutional neural networks 18

[2 min] Reveal the four positions. Step 1 is the top-left value, 0. Step 2 moves one column right: 2·1 + 0·0 + 1·0 + 3·(−1) = −1. Step 3 moves down a row: 0·1 + 1·0 + 2·0 + 1·(−1) = −1. Step 4: 1·1 + 3·0 + 1·0 + 0·(−1) = 1. The output map is [[0, −1], [−1, 1]].

The same kernel weights are used in all four steps; only the input patch changes. That is weight sharing made concrete. In a real layer the window moves across thousands of positions and there are many kernels, but each output value is still this four-term (or nine-term, or 27-term) weighted sum plus a bias.

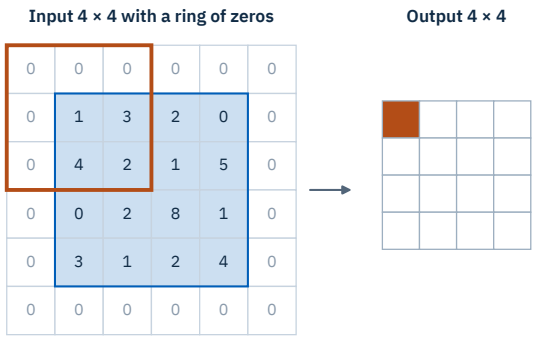
Common error: flipping the kernel because a signal-processing course defined convolution that way. For learned kernels it does not matter, but for hand-set kernels such as edge detectors, flipping reverses the sign of the response.

Zero padding keeps border pixels in play

CONVOLUTION MECHANICS

Zero padding keeps border pixels in play

- **Padding** adds a border so windows can centre on edge pixels.
- With K = 3, P = 1 and stride 1, the output keeps the input size.
- Border outputs mix real values with artificial zeros.



K = 3, P = 1, S = 1: (4 + 2·1 − 3)/1 + 1 = 4

The corner window holds 5 padding zeros and 4 real values

Schematic with exact output-size arithmetic; padding width 1.

Neural Networks / Day 4 · Convolutional neural networks 19

[3 min] Without padding a 3 × 3 kernel cannot centre on the first or last row or column, so each layer would shrink the map by 2 in each direction. Padding by one pixel of zeros restores the size: (4 + 2·1 − 3)/1 + 1 = 4. This is often called "same" padding; PyTorch accepts padding="same" for stride 1.

The orange window at the corner contains five padding zeros and only four real values. Border outputs therefore see partly artificial context, and they can behave differently from interior outputs. That is harmless for many photographs but can matter when the object of interest sits at the edge of a patch, which happens constantly with tiled logs and seismic.

PyTorch also offers padding_mode values "reflect", "replicate" and "circular". Choose by physics: circular for wrap-around axes, reflect or replicate when zero is a meaningful value (a thickness of zero is not "no signal"). The next slide shows the casing case.

Azimuth wraps around, depth does not

CONVOLUTION MECHANICS

Azimuth wraps around, depth does not

```python
import torch
import torch.nn.functional as F
from torch import nn

x = torch.randn(16, 1, 32, 36)  # B, C, depth, azimuth
x = F.pad(x, (1, 1, 0, 0), mode="circular")
conv = nn.Conv2d(1, 8, kernel_size=3, padding=(1, 0))
print(x.shape, conv(x).shape)
```

OUTPUT

```
torch.Size([16, 1, 32, 38]) torch.Size([16, 8, 32, 36])
```

- `F.pad` wraps only the last axis, azimuth.
- `padding=(1, 0)` zero-pads depth only.
- `padding_mode="circular"` would wrap depth too.

Neural Networks / Day 4 · Convolutional neural networks 20

[3 min] This answers question 3 of the tensor exercise. The pad tuple lists the last axis first: (left, right) for azimuth, then (top, bottom) for depth. Circular mode copies sector 35 in front of sector 0 and sector 0 after sector 35, so a pit that straddles 0° is seen whole. The azimuth axis grows from 36 to 38; the convolution then uses padding=(1, 0), meaning one row of zeros in depth and none in azimuth, and returns 32 × 36 again.

Why not simply set padding_mode="circular" on Conv2d? Because it wraps every spatial axis, including depth, which would make the top of a patch a neighbour of its bottom: physically wrong.

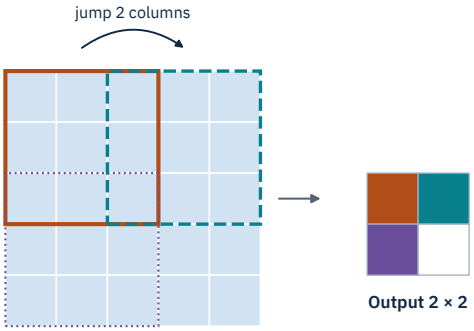
Check yourself with a small tensor: padding [0, 1, 2, 3, 4, 5] circularly gives [5, 0, 1, 2, 3, 4, 5, 0]. Borehole images have the same geometry and need the same treatment.

Stride skips positions and shrinks the map

CONVOLUTION MECHANICS

Stride skips positions and shrinks the map

- **Stride** S moves the window S pixels at each step.
- S = 2 roughly halves height and width.
- Neighbouring outputs now sit S input pixels apart.



Input 5 × 5

K = 3, S = 2, P = 0: ⌊(5 − 3)/2⌋ + 1 = 2

Schematic; three of the four window positions shown.

[2 min] With stride 2 the window jumps two columns between positions: the orange window starts at column 0, the dashed teal one at column 2, and the dotted purple one two rows down. On a 5 × 5 input with a 3 × 3 kernel and no padding there are only two positions per axis: floor((5 − 3)/2) + 1 = 2.

Stride is the standard way to downsample inside a convolution: it reduces computation for all later layers and increases how much of the input each later unit sees. The cost is resolution. Fine detail between the sampled positions is not lost from the input, but the output grid can no longer represent it directly.

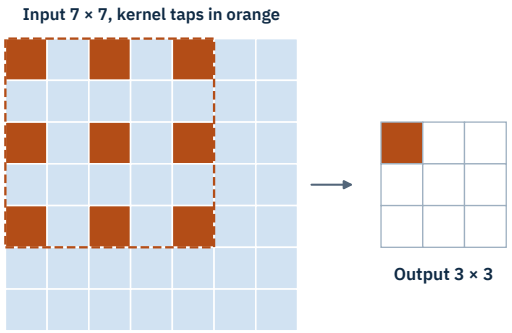
Ask: in a segmentation network that must outline a thin fault, what happens after four stride-2 layers? The map is 16 times coarser, which is why segmentation architectures add decoders and skip connections (Part 6).

Dilation widens a kernel's reach, not its weights

CONVOLUTION MECHANICS

Dilation widens a kernel's reach, not its weights

- **Dilation** D spaces the kernel taps D pixels apart.
- Nine weights cover a 5 × 5 area when D = 2.
- It adds context without downsampling: segmentation, seismic, 1-D signals.



K = 3, D = 2: taps span D(K – 1) + 1 = 5 pixels

Still 9 weights; (7 – 5)/1 + 1 = 3 outputs per row

Schematic; dilation 2, no padding, stride 1.

Neural Networks / Day 4 · Convolutional neural networks 22

[2 min] Dilation inserts gaps between kernel taps. With D = 2 the nine orange taps sit two pixels apart and cover a 5 × 5 region: the effective size is D(K – 1) + 1 = 2·2 + 1 = 5. The number of weights is still nine. On a 7 × 7 input without padding, the output is 7 – 5 + 1 = 3 per axis.

Why use it? Stacking dilated layers with increasing D grows the receptive field quickly while keeping full resolution, which suits dense prediction such as segmentation, and 1-D signals where long context matters. Temporal convolutional networks on Day 5 use exactly this idea along time.

Caution: dilated kernels skip pixels, so a stack with a single dilation rate can miss fine detail in a checkerboard-like pattern; practical designs mix rates.

Output size follows kernel, stride, padding and dilation

CONVOLUTION MECHANICS

Output size follows kernel, stride, padding and dilation

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - D(K-1) - 1}{S} + 1 \right\rfloor$$

- K kernel size, S stride, P padding, D dilation; the same rule gives the width.
- H_{in} = 32: K = 3, P = 1 keeps 32; adding S = 2 gives 16; K = 5, P = 0 gives 28.
- The number of output channels is the number of filters.

[3 min] This is the formula in the PyTorch Conv2d documentation. Check the examples with the class. K = 3, P = 1, D = 1, S = 1: (32 + 2 − 2 − 1)/1 + 1 = 32. With S = 2: (32 + 2 − 2 − 1)/2 + 1 = 16.5, floored to 16. K = 5, P = 0: (32 − 4 − 1)/1 + 1 = 28. A dilated case: K = 3, D = 2, P = 2 gives (32 + 4 − 4 − 1)/1 + 1 = 32.

The floor matters: when the window does not fit evenly, the last partial position is dropped, so some border pixels never reach the output. The formula assumes symmetric integer padding; frameworks also offer "same" conventions.

Channels are not in the formula at all: they are set by the number of filters you ask for. Pooling layers follow the same size rule with their own kernel and stride.

Predict the shape before you run the layer

PRACTICE

4 min · Individual

Predict the shape before you run the layer

Input: B × 3 × 32 × 32 in PyTorch. Layer: `Conv2d(3, 16, kernel_size=3, stride=2, padding=1)`.

1. State the output shape.
2. Count the trainable parameters, including biases.
3. What changes, and what does not, if the input becomes 64 × 64?

Neural Networks / Day 4 · Convolutional neural networks 24

[4 min individual, 1 min check]

1. Height and width: floor((32 + 2·1 − 3)/2) + 1 = floor(15.5) + 1 = 16. Output: B × 16 × 16 × 16 (batch, 16 channels, 16 × 16).

2. Parameters: 16 filters × (3 × 3 × 3 weights + 1 bias) = 16 × 28 = 448.

3. At 64 × 64 the output becomes B × 16 × 32 × 32. The parameter count stays 448, because the same filters are reused at every position. Activation memory and computation for this layer grow four-fold, since there are four times as many output positions.

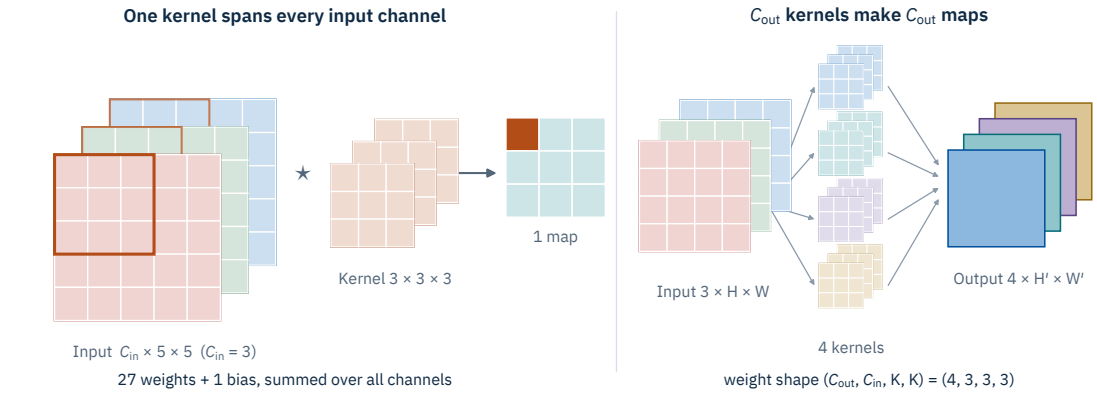
Common errors: dividing 32 by 2 and forgetting the floor in odd cases; counting 3 × 3 = 9 weights per filter and forgetting the three input channels; forgetting biases. Ask learners to verify in Lab 1 by printing sum(p.numel() for p in layer.parameters()).

Each kernel spans all input channels and makes one map

CONVOLUTION MECHANICS

Each kernel spans all input channels and makes one map

A Conv2d weight tensor has shape (C_{out}, C_{in}, K, K) and C_{out} (C_{in} K^2 + 1) parameters.



Schematic; one bias per output channel.

Neural Networks / Day 4 · Convolutional neural networks 25

[3 min] Left: with a three-channel input, one kernel is a 3 × 3 × 3 block. At each position it multiplies all 27 values under it, sums them, adds one bias and writes one number. Sliding it over the image produces one feature map. The channels are summed, not kept separate; that is why a filter can respond to "red above green" combinations.

Right: a layer with four filters produces four maps, stacked as the output tensor's channel axis. The weight tensor has shape (4, 3, 3, 3), with four biases, so 4 × (27 + 1) = 112 parameters. The next layer's kernels then span these four channels.

Ask: what is C_in for the second convolution of the compact CNN? It equals the number of filters in the first layer, 16. This is how channel counts chain through a network. Common error: thinking each input channel gets its own separate output map; that is a depthwise convolution (groups = C_in), used by efficient architectures, not the default.

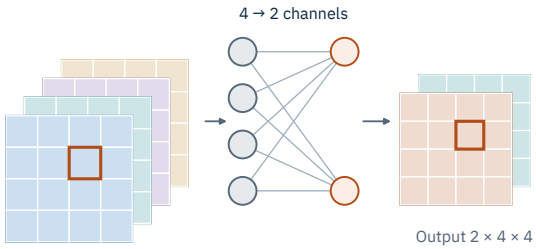
A 1 × 1 convolution mixes channels at each pixel

CONVOLUTION MECHANICS

A 1 × 1 convolution mixes channels at each pixel

- A **1 × 1 convolution** is a dense layer applied at every pixel.
- It changes the channel count cheaply: C_{out} (C_{in} + 1) parameters.
- Bottleneck blocks use it before costly 3 × 3 layers.

The same small dense layer at every pixel



Parameters: 2 × (4 + 1) = 10; H and W unchanged

Schematic: four channels mapped to two at every pixel.

Neural Networks / Day 4 · Convolutional neural networks 26

[3 min] A 1 × 1 kernel looks at a single pixel, but across all channels. At each position it takes the C_in-vector of channel values and applies the same small linear map (plus bias) to produce C_out values: exactly a dense layer shared across positions. Height and width do not change. Here 4 → 2 channels costs 2 × (4 + 1) = 10 parameters.

Why it matters: 3 × 3 convolutions on many channels are expensive, because cost scales with C_in × C_out × 9. Inception and ResNet bottleneck blocks first reduce channels with a 1 × 1 layer, apply the 3 × 3 layer on fewer channels, then expand again. Projection shortcuts in residual networks (Part 4) are also 1 × 1 convolutions.

In seismic work, a 1 × 1 convolution over stacked attributes is a learned per-sample combination of attributes, a useful mental model for what the first layer of a multi-attribute network does.

Count the parameters before you build

PRACTICE

5 min Pairs

Count the parameters before you build

Count weights and biases for each layer:

1. `Conv2d(64, 128, kernel_size=3, padding=1)`
2. `Conv2d(64, 16, kernel_size=1)`
3. Two stacked 3 × 3 layers (64 → 64 channels) versus one 5 × 5 layer (64 → 64). Which sees further?

Neural Networks / Day 4 · Convolutional neural networks 27

[5 min in pairs, then reveal the next slide]

1. 128 × (64 × 3 × 3 + 1) = 128 × 577 = 73,856.

2. 16 × (64 × 1 × 1 + 1) = 16 × 65 = 1,040.

3. One 3 × 3 layer from 64 to 64 channels: 64 × (64 × 9 + 1) = 64 × 577 = 36,928; two of them: 73,856. One 5 × 5 layer: 64 × (64 × 25 + 1) = 64 × 1,601 = 102,464. Both see a 5 × 5 input region: the second 3 × 3 layer sees 3 × 3 outputs of the first, each of which saw 3 × 3 inputs, so the field is 3 + (3 − 1) = 5.

It is a coincidence of these numbers that answers 1 and 3 match; do not let learners over-read it.

Common errors: forgetting the input channels in the kernel; adding biases per weight rather than per filter; thinking the 5 × 5 layer sees further because its kernel is bigger.

Two 3 × 3 layers see as far as one 5 × 5 layer



[2 min] This is the design argument behind VGG, one of the historical architectures in Part 4: replace large kernels with stacks of small ones. The stack uses about 28% fewer parameters (73,856 versus 102,464) for the same receptive field, and the ReLU between the two layers lets it represent functions a single linear 5 × 5 filter cannot.

The trade-off is depth: more layers mean more sequential computation and more activations to store for backpropagation, and very deep plain stacks become hard to optimize, which motivates residual connections later today.

Transition: before we stack layers, one statement to anchor what convolution assumes.

Convolution is a dense layer with two assumptions

CONVOLUTION MECHANICS

Convolution is a dense layer with two assumptions

Nearby values belong together, and useful patterns recur across positions.

Right for images and ordered signals; wrong for shuffled table columns.

Neural Networks / Day 4 · Convolutional neural networks 29

[1 min] A convolution layer is a dense layer with most of its weights forced to zero (locality) and the remaining weights forced to be equal across positions (sharing). Every benefit of CNNs, from fewer parameters to shift equivariance, follows from these two constraints, and so does every failure when the constraints do not match the data.

Ask for one dataset in the room where both assumptions hold (a borehole image, a thermal drone image) and one where they do not (a table of completion parameters per well). Then move to Part 3, where stacking these layers produces feature maps.

Feature maps and a compact CNN



[About 45 minutes for this part.] We move from one layer to a stack. First we look at what filters respond to, hand-set and learned, then at how far a stacked unit can "see", then at the ways networks reduce resolution, and at what shifting the input does to the output. The part ends with the exact model and shape ledger used in Lab 1, so learners walk into the lab having predicted every shape.

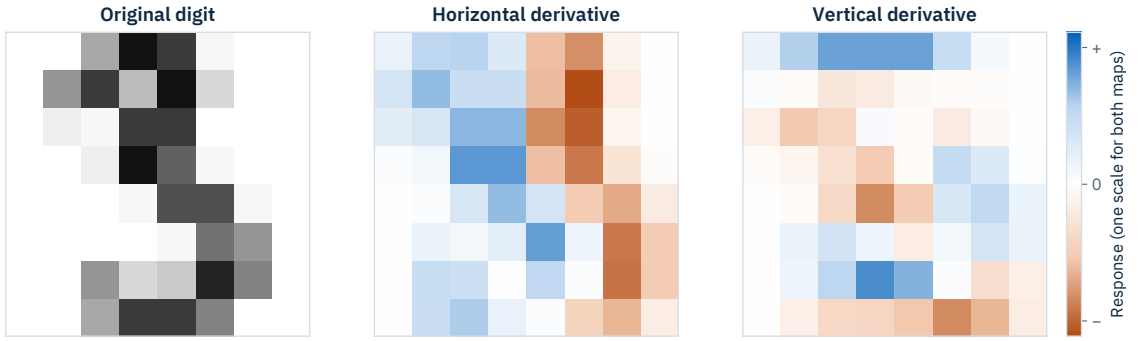
Keep the receptive-field exercise: it is the most transferable skill of the part, because it converts a network design into a physical length on a log or a seismic section.

Filters respond to local structure

FEATURE MAPS AND A COMPACT CNN

Filters respond to local structure

Sobel kernels respond where intensity changes; blue and orange mark opposite signs.



Fixed Sobel kernels applied to scikit-learn digit 3 with zero padding; hand-specified, not learned; one scale centred on zero.

[2 min] Point to positive and negative responses. The horizontal-derivative kernel [[−1, 0, 1], [−2, 0, 2], [−1, 0, 1]] responds strongly where ink starts or stops as you move left to right, that is, along vertical strokes; the vertical-derivative kernel responds along horizontal strokes. Blue marks a positive response (intensity rising to the right, or downwards for the vertical kernel) and orange a negative one: entering a stroke versus leaving it. White is zero, the same on both maps, as on the learned filters of the next slide.

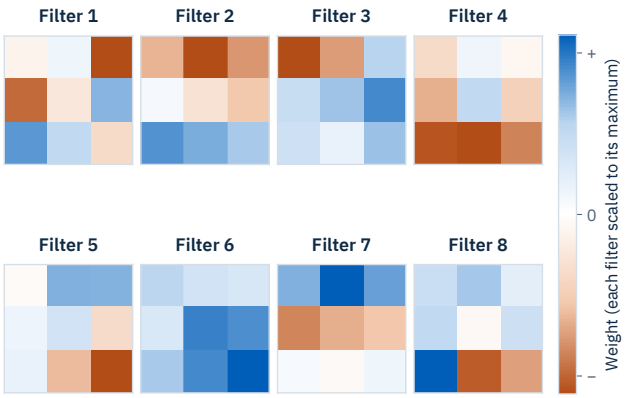
These are hand-specified teaching filters from classical image processing. Learned first-layer filters can detect similar simple local patterns, while deeper feature maps combine them into larger context. Avoid claiming that every channel corresponds to a named human concept; next slide shows what a small network actually learned.

Learned first-layer filters act as local detectors

FEATURE MAPS AND A COMPACT CNN

Learned first-layer filters act as local detectors

- A tiny CNN trained on 1,437 digits learned these eight 3 × 3 kernels.
- Several resemble oriented edge or stroke detectors.
- Not every filter has a human name; another seed gives different filters.



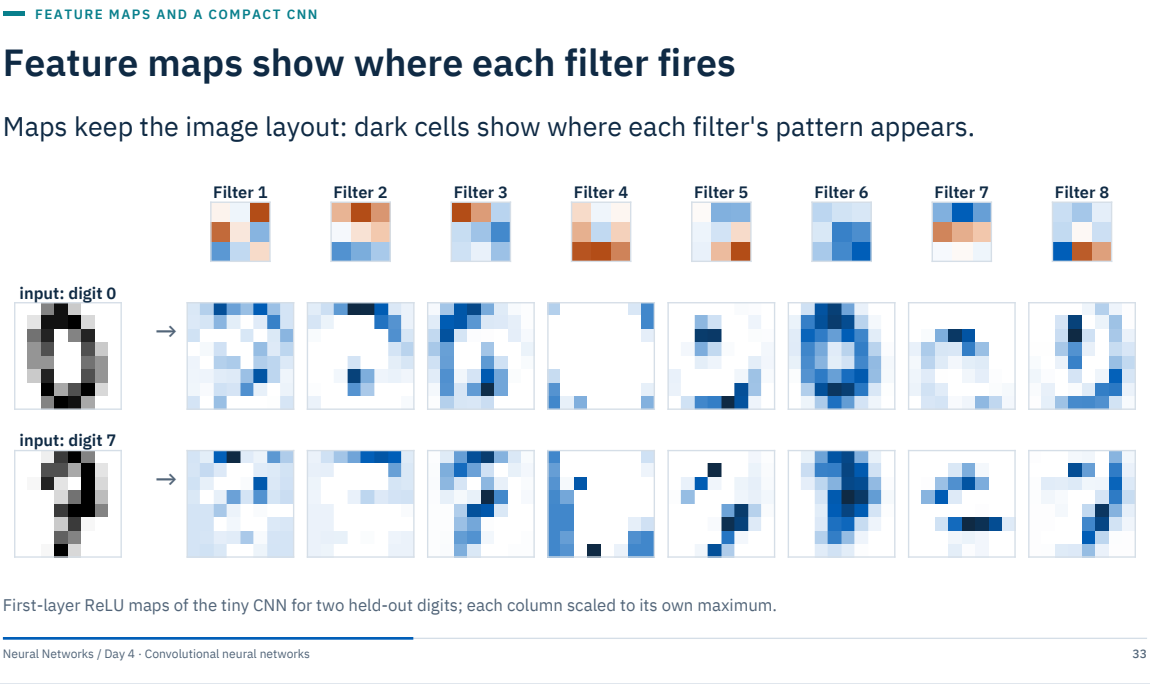
Tiny CNN (1,418 parameters) trained on scikit-learn digits, seed 0, CPU; 95.0% accuracy on 360 held-out digits.

[3 min] This network has the same shape as the compact CNN you will meet shortly: conv 3 × 3 with 8 filters, ReLU, 2 × 2 max pooling, conv 3 × 3 with 16 filters, ReLU, global average pooling and a linear layer: 1,418 parameters in total. It was trained for 60 epochs with Adam on 1,437 digits and scores 95.0% (342 of 360) on the held-out digits.

Each tile shows one first-layer kernel, scaled to its own largest weight: blue positive, orange negative. Filter 4 has a negative bottom row under a positive centre (a horizontal edge), filter 7 a positive top row over a negative middle row. Others are harder to name.

Two honest caveats. Filters are not unique: a different random seed gives a different but equally good set, because the network can permute and mix them. And "looks like an edge detector" is an interpretation; the next slide checks what the filters actually respond to.

Feature maps show where each filter fires

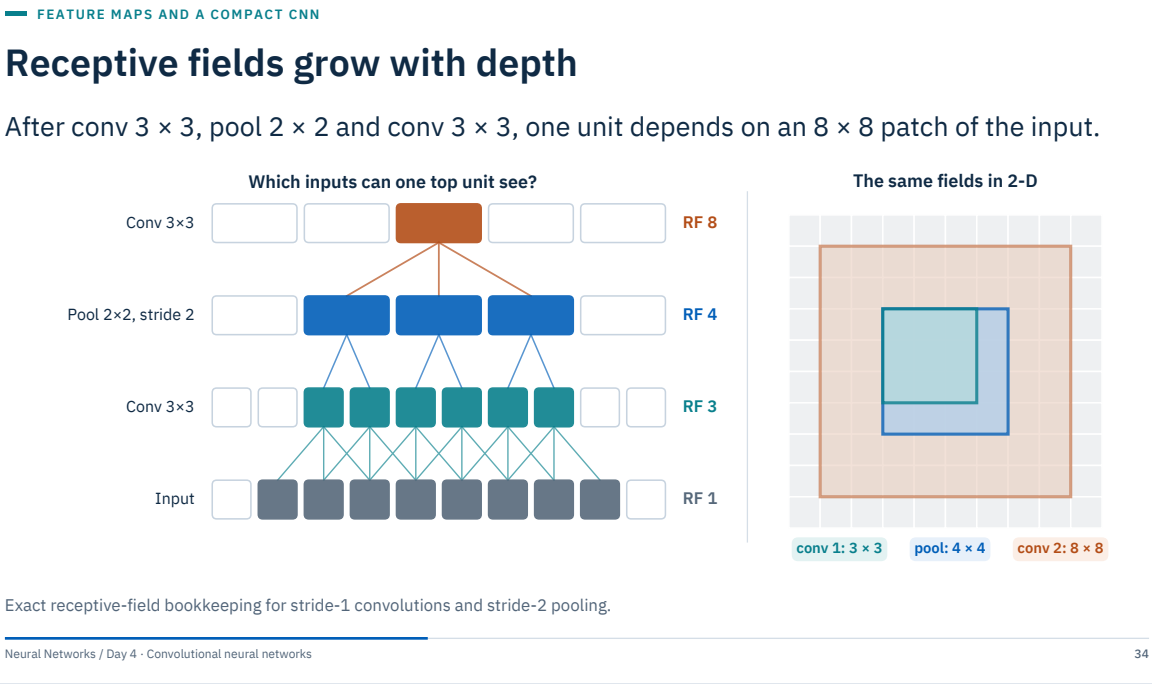


[3 min] Each column is one filter (shown on top) applied to two held-out digits, a 0 and a 7, after the ReLU. Because convolution preserves layout, each feature map is itself a small image: filter 3 lights up along the left stroke of both digits, filter 7 along a horizontal band, filter 6 on the dense body of the 0. Several maps are almost empty for one digit and active for the other; that difference is what later layers use.

This is the concrete meaning of "a CNN turns an image into a stack of feature maps": 1 input channel became 8 channels, each answering "is my pattern here?" at every position. The second layer then combines these 8 maps with its own 3 × 3 × 8 kernels.

Ask: why are the maps the same size as the input? Because of padding 1 with a 3 × 3 kernel and stride 1, from Part 2.

Receptive fields grow with depth



[3 min] Left: follow one top unit down. The second convolution reads 3 neighbouring pooled units; each pooled unit covers 2 first-layer units, so together they cover 6; each of those reads 3 inputs, and the union is 8 inputs. Right: the same fields in 2-D, 3 × 3 after the first convolution, 4 × 4 after pooling and 8 × 8 after the second convolution.

Why it matters: a unit can only respond to structure inside its receptive field. If a defect or a fault offset spans 30 pixels, a network whose top units see 8 pixels must rely on the head to combine many local votes, or it needs more depth, more pooling or dilation.

Caveat: this is the theoretical field. The effective influence of pixels inside it is uneven, concentrated near the centre, so the practical field is often smaller.

A receptive-field ledger tracks jump and size

FEATURE MAPS AND A COMPACT CNN

A receptive-field ledger tracks jump and size

Layer	Kernel / stride	Jump j	Receptive field r
Input	–	1	1
Conv 1	3 / 1	1	3
Pool	2 / 2	2	4
Conv 2	3 / 1	2	8

Each layer applies $r \leftarrow r + (K - 1)\, j$, then $j \leftarrow j \cdot S$, with kernel K and stride S.

[2 min] Recurrence: the new receptive field is r_new = r_old + (K − 1) × jump, where the jump is the distance in input pixels between neighbouring units of the previous layer; then jump_new = jump × stride. Walk the table: Conv 1 gives 1 + 2·1 = 3; Pool gives 3 + 1·1 = 4 and the jump becomes 2; Conv 2 gives 4 + 2·2 = 8.

Dilation increases the effective kernel: use K_eff = D(K − 1) + 1 in the same formula. The empirical influence of pixels may be smaller or uneven within the theoretical field.

The jump column is also the spacing of the output grid in input pixels, which tells you how finely a layer can localize something: after this stack each unit is 2 input pixels from its neighbour.

Extend the receptive-field ledger

PRACTICE

4 min Pairs

Extend the receptive-field ledger

Continue the table after Conv 2:

1. Add a pool 2 / 2, then a conv 3 / 1. Give j and r after each.
2. How many stride-1 3 × 3 convolutions alone would cover at least the same field?
3. Each depth row spans 0.1 m. How tall is that field in metres?

[4 min in pairs, 2 min answers]

1. Pool 2 / 2: r = 8 + (2 − 1) × 2 = 10; j = 2 × 2 = 4. Conv 3 / 1: r = 10 + (3 − 1) × 4 = 18; j stays 4.

2. A stack of n stride-1 3 × 3 convolutions has r = 1 + 2n. We need 1 + 2n ≥ 18, so n = 9 (field 19); n = 8 gives only 17. An exact field of 18 is impossible because the stack always gives odd sizes. The pooling version reaches 18 with three convolutions and two pooling layers instead of nine convolutions, at the price of a coarser output grid (jump 4).

3. 18 rows × 0.1 m = 1.8 m of depth. On an azimuth axis with 10° sectors, the same 18 pixels would be 180°: half the circumference, a reminder that square kernels do not see square physical areas.

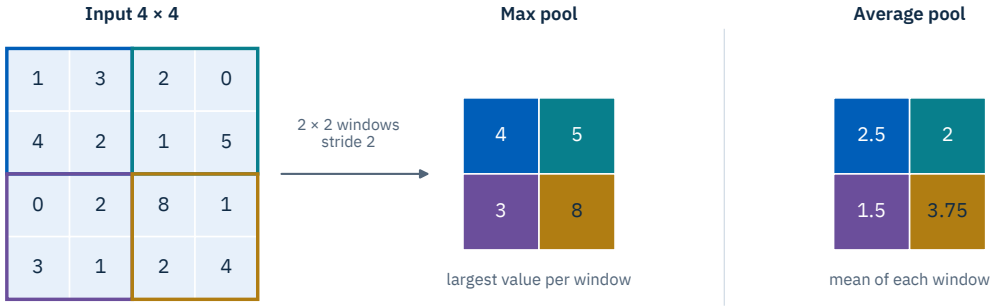
Common errors: forgetting to multiply by the jump; updating the jump before the field.

Pooling summarizes neighbourhoods

FEATURE MAPS AND A COMPACT CNN

Pooling summarizes neighbourhoods

Max pooling keeps the strongest response in each window; average pooling keeps the mean.



Exact max and average pooling on one 4 × 4 matrix, 2 × 2 windows, stride 2; colours link each window to its output.

Neural Networks / Day 4 · Convolutional neural networks 37

[2 min] Have learners compute one output cell from each operation. Top-left window [[1, 3], [4, 2]]: the maximum is 4 and the mean is 10/4 = 2.5. Bottom-right window [[8, 1], [2, 4]]: the maximum is 8 and the mean is 15/4 = 3.75.

Max pooling keeps the largest local value, useful when the question is "does this pattern occur somewhere in this window?". Average pooling keeps the local mean. Both halve the resolution here, both have no trainable parameters, and both discard information: after max pooling you cannot tell where in the window the 8 was, or what the other three values were.

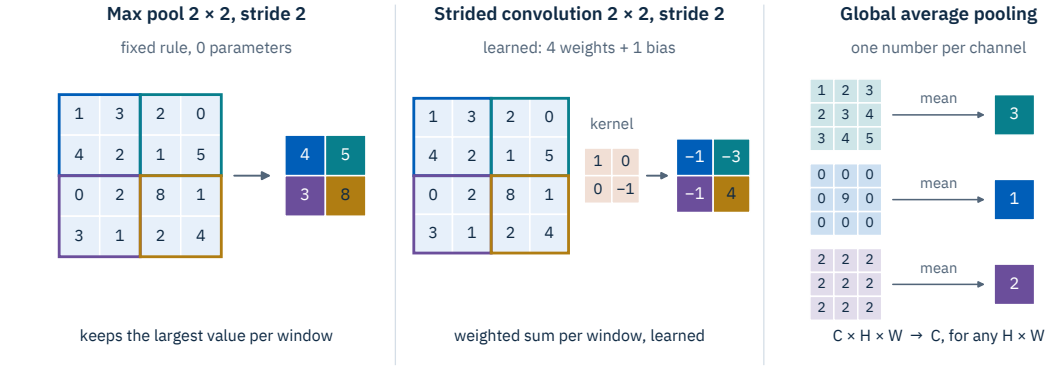
They do not guarantee complete invariance to shifts, which we test two slides from now.

Strided convolution and global averaging also downsample

FEATURE MAPS AND A COMPACT CNN

Strided convolution and global averaging also downsample

Modern CNNs often use strided convolutions and end with global average pooling.



Exact arithmetic; the strided convolution uses kernel [[1, 0], [0, −1]] and zero bias.

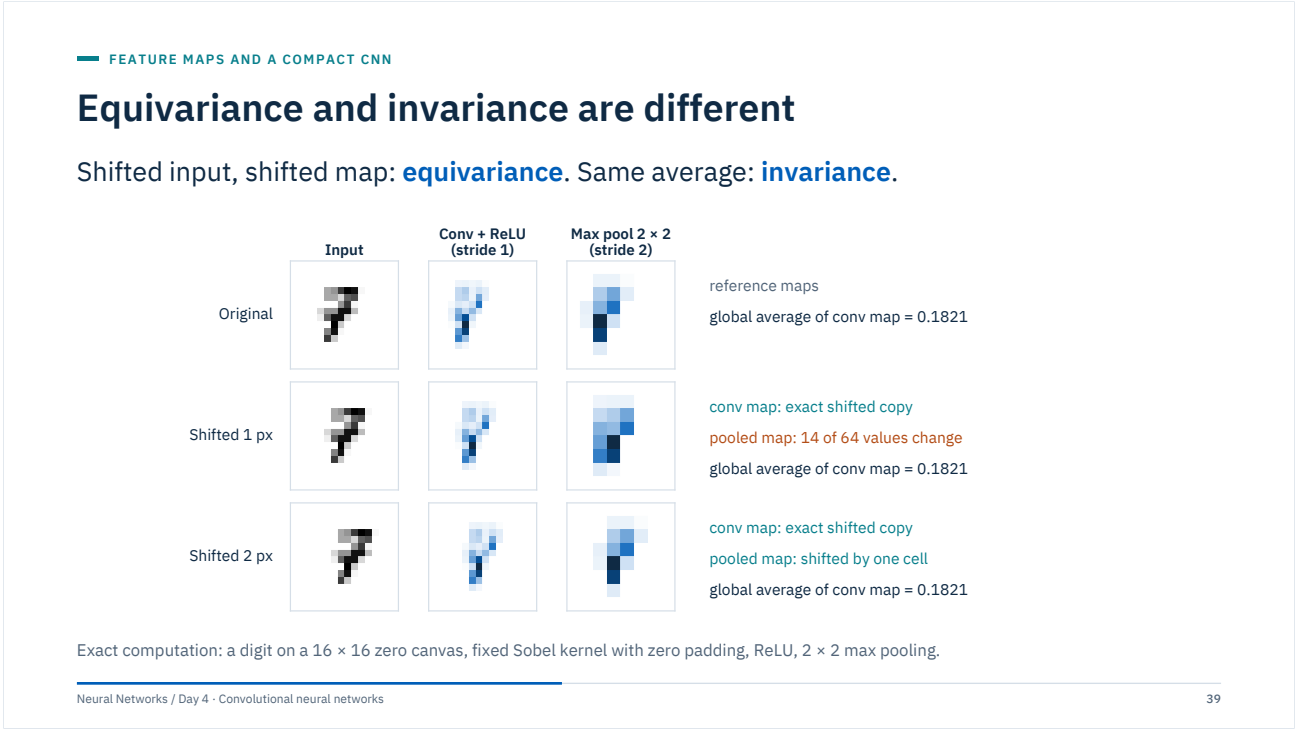
Neural Networks / Day 4 · Convolutional neural networks 38

[3 min] Left: max pooling, a fixed rule. Middle: a 2 × 2 convolution with stride 2 visits the same four windows, but computes a weighted sum with learned weights. With kernel [[1, 0], [0, −1]]: the top-left window gives 1 − 2 = −1; top-right 2 − 5 = −3; bottom-left 0 − 1 = −1; bottom-right 8 − 4 = 4. If all four weights were 0.25, this layer would be average pooling: pooling is a special, fixed case of a strided convolution.

Right: global average pooling (GAP) collapses each whole feature map to its mean, here 3, 1 and 2. The output has one number per channel, whatever the height and width, which removes the large flatten-and-dense head and lets the same network accept different input sizes in principle.

Ask: what does GAP throw away? All spatial layout. That is fine for "is there a defect in this patch?" and wrong for "where is it?".

Equivariance and invariance are different



[4 min] Avoid the common blanket claim that CNNs are translation invariant. Here a digit is shifted 1 and 2 pixels to the right. The stride-1 convolution map is an exact shifted copy each time: that is equivariance. Its global average is 0.1821 in all three rows: that is invariance of the summary. But the max-pooled map after a 1-pixel shift is not a shifted copy: 14 of its 64 values change. After a 2-pixel shift, a multiple of the stride, it is again an exact one-cell shift.

So striding and pooling break exact equivariance for shifts that are not multiples of the stride; boundaries, padding and final dense layers also affect behaviour. A spatially meaningful output such as segmentation needs equivariance rather than invariance.

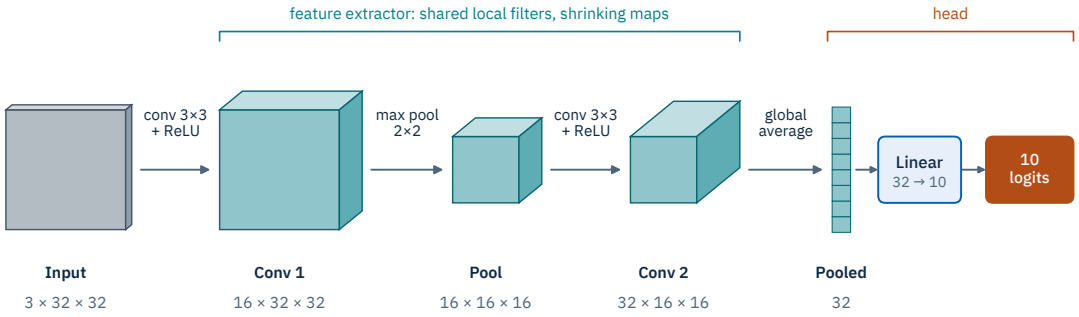
Practical rule: test the actual shifts and distortions expected in deployment, for example a depth mis-tie of a few samples between logging runs, rather than assuming the architecture handles them.

A CNN alternates feature extraction and downsampling

FEATURE MAPS AND A COMPACT CNN

A CNN alternates feature extraction and downsampling

Spatial size shrinks while channels grow; a small head turns pooled features into class logits.



Schematic of the compact CNN on the next slides; shapes are C × H × W for one image.

Neural Networks / Day 4 · Convolutional neural networks 40

[2 min] This is the picture behind almost every classification CNN: blocks of convolution and nonlinearity, interleaved with downsampling, so that later layers see larger regions with more channels. Here an RGB 32 × 32 image becomes 16 maps at full resolution, is pooled to 16 × 16, becomes 32 maps, and global averaging reduces each map to one number. A linear layer maps the 32 numbers to 10 logits, one per class.

The teal bracket is the feature extractor: reusable, and the part a pretrained backbone replaces in transfer learning. The orange bracket is the task-specific head. Keep this split in mind for Part 5.

Use this model for a CIFAR-style 32-pixel RGB input. It is deliberately small, a teaching baseline rather than a state-of-the-art model.

A shape ledger records every tensor in the model

FEATURE MAPS AND A COMPACT CNN

A shape ledger records every tensor in the model

Operation	Output for one batch	Purpose
Input	B × 3 × 32 × 32	RGB images
Conv 3 × 3, 16 filters, padding 1	B × 16 × 32 × 32	Local features
ReLU, max pool 2 × 2	B × 16 × 16 × 16	Nonlinearity, downsampling
Conv 3 × 3, 32 filters, padding 1, ReLU	B × 32 × 16 × 16	Feature combinations
Global average pool, linear	B × 10	Class logits

[2 min] Ask learners to verify each row with the output-size formula before looking: 32 + 2 – 3 + 1 = 32; pooling halves 32 to 16; the second convolution keeps 16; global averaging leaves 32 numbers per image, and the linear layer maps them to 10.

Global average pooling produces one number per channel before the final linear layer. This avoids a large flatten-and-dense parameter count and accepts different spatial sizes in principle, though deployment still needs consistent input preprocessing.

Write a ledger like this for every model you build, including the batch axis. It is the fastest way to find the mismatch when a shape error appears, and reviewers can check it without running code.

The compact CNN fits in one Sequential block

FEATURE MAPS AND A COMPACT CNN

The compact CNN fits in one Sequential block

```
from torch import nn

model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(), nn.Linear(32, 10),
)
criterion = nn.CrossEntropyLoss()
```

No softmax before `CrossEntropyLoss`: it takes raw logits and integer class labels 0 to 9.

Neural Networks / Day 4 · Convolutional neural networks 42

[2 min] Input must be B, C, H, W for this PyTorch example. Targets are integer class indices from 0 to 9. No softmax appears before CrossEntropyLoss, because the loss applies log-softmax internally; adding a softmax layer squashes the logits twice and slows learning.

AdaptiveAvgPool2d(1) is global average pooling: whatever the incoming height and width, it outputs 1 × 1 per channel, and Flatten turns B × 32 × 1 × 1 into B × 32.

Run a dummy batch through the model to check shape before downloading or training on a full dataset; the next slide does exactly that. Lab 1's notebook model, SimpleCNN, uses 32 and 64 filters and a flatten-plus-dense head instead of GAP; compare both in the lab.

Run a dummy batch before any training

FEATURE MAPS AND A COMPACT CNN

Run a dummy batch before any training

```python
import torch

x = torch.randn(8, 3, 32, 32)
for layer in model:
    x = layer(x)
    print(f"{layer.__class__.__name__:<18} {tuple(x.shape)}")
print(sum(p.numel() for p in model.parameters()))
```

OUTPUT

```
Conv2d             (8, 16, 32, 32)
ReLU               (8, 16, 32, 32)
MaxPool2d          (8, 16, 16, 16)
Conv2d             (8, 32, 16, 16)
ReLU               (8, 32, 16, 16)
AdaptiveAvgPool2d  (8, 32, 1, 1)
Flatten            (8, 32)
Linear             (8, 10)
5418
```

Neural Networks / Day 4 · Convolutional neural networks 43

[2 min] A random batch of 8 images costs nothing and checks the whole ledger. Iterating over an nn.Sequential gives each layer in order; printing after each one catches the classic error of a flatten size that does not match the next dense layer. The output matches the ledger slide row by row.

The parameter count, 5,418, is 448 + 4,640 + 330: the first convolution 16 × (3 × 9 + 1), the second 32 × (16 × 9 + 1), and the linear layer 10 × (32 + 1). Pooling, ReLU and flatten have no parameters.

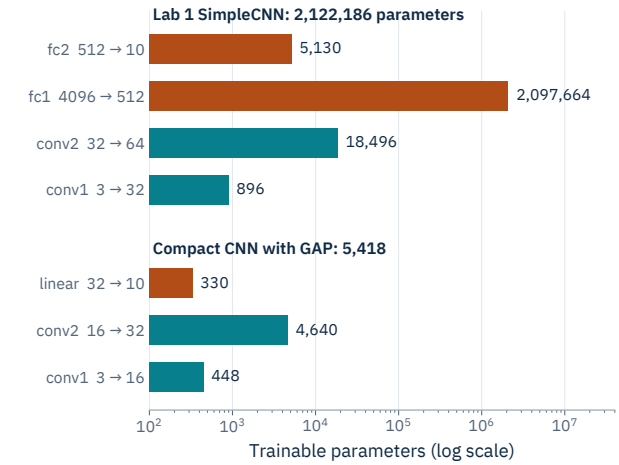
For models written as classes rather than Sequential, register forward hooks or print inside forward; the Lab 1 notebook uses forward hooks. Do this in every lab before the first training loop.

Parameters concentrate in dense heads

FEATURE MAPS AND A COMPACT CNN

Parameters concentrate in dense heads

- Lab 1's SimpleCNN has 2,122,186 parameters; 98.8% sit in one dense layer.
- Global average pooling shrinks the head to 330 parameters.
- Fewer parameters is not automatically better: compare on validation data.



Exact counts (weights and biases), checked with PyTorch.

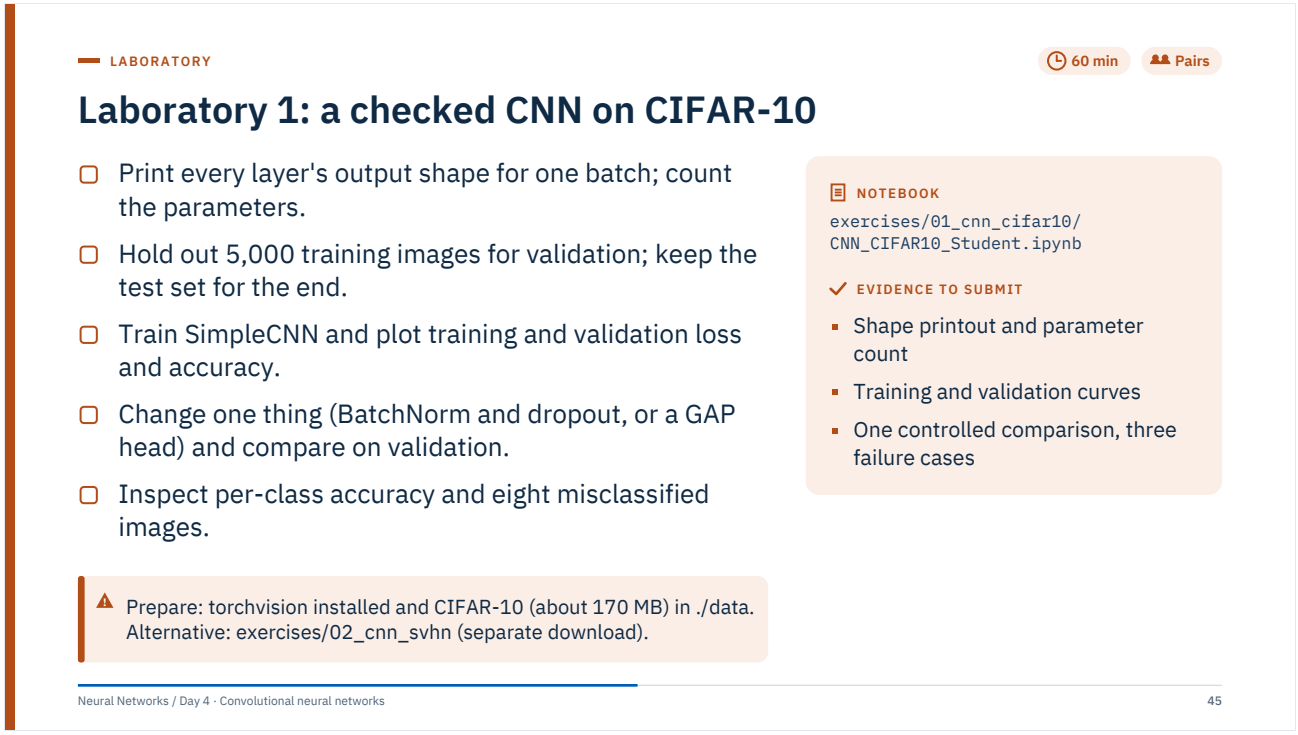
Neural Networks / Day 4 · Convolutional neural networks 44

[3 min] SimpleCNN in the Lab 1 notebook flattens a 64 × 8 × 8 tensor (4,096 numbers) into a dense layer of 512 units: 4,096 × 512 + 512 = 2,097,664 parameters, 98.8% of the model's 2,122,186. The two convolutions together have fewer than 20,000.

The compact GAP model has 5,418 parameters in total. That does not make it better or worse by itself: the flatten head keeps spatial layout information that GAP discards, and a larger head can overfit or can help, depending on the data. The lab asks you to compare on validation data rather than argue from parameter counts.

The log scale is essential here: on a linear axis the convolution bars would be invisible. Transition: now build, check and train.

Laboratory 1: a checked CNN on CIFAR-10



[60 min; debrief at 13:30] Suggested timing: 0–10 min setup and shape check; 10–20 validation split; 20–45 training runs; 45–60 comparison and error inspection.

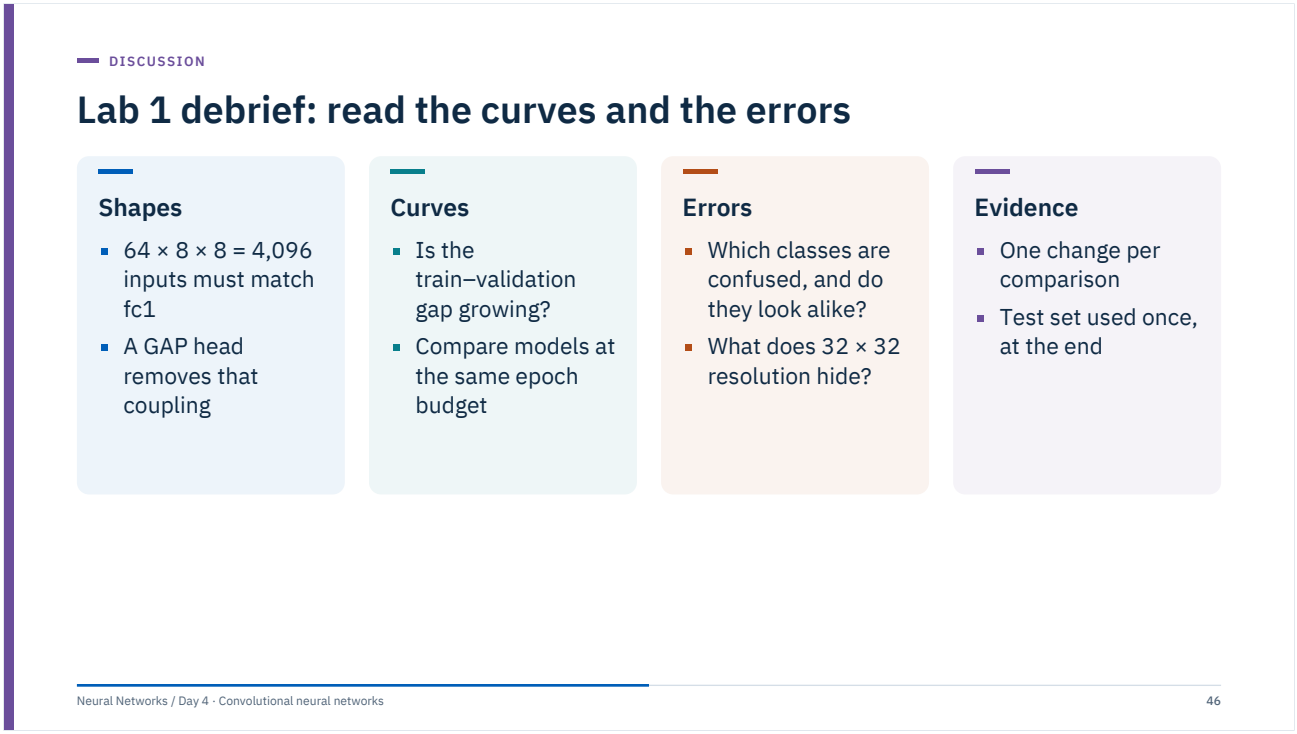
Validation split: the notebook holds out 5,000 training images with train_set, val_set = torch.utils.data.random_split(full_train_dataset, [45000, 5000], generator=torch.Generator().manual_seed(42)). Each model keeps its epoch with the best validation accuracy; per-class accuracy, the misclassified images and the model comparison use validation data; the chosen model meets the test set once, in Section 8. For CIFAR-10, whose images are independent photographs, a random split is acceptable; contrast this with the casing data in Lab 2B.

CPU budget: both models train on the same fixed 20,000-image subset of the training split for 10 epochs (N_TRAIN, NUM_EPOCHS): about 3–5 minutes on a laptop CPU, 1–2 minutes on Apple silicon or a GPU. Raise N_TRAIN to 45,000 if time allows; the comparison stays valid as long as both models get the same budget. The notebook's comparison model, ImprovedCNN, is SimpleCNN plus BatchNorm and dropout, with everything else unchanged.

Common errors: normalizing with the wrong channel statistics; forgetting model.eval() during validation, so dropout and BatchNorm behave as in training; changing two things at once; a DataLoader that hangs in a notebook (set num_workers=0).

SVHN alternative: exercises/02_cnn_svhn follows the same protocol (shape check, validation split, best epoch, one test evaluation) with a three-block CNN; its controlled comparison is augmentation and its error analysis a confusion matrix. Its augmentation uses only shifts of up to 2 pixels and colour jitter; it deliberately leaves out horizontal flips, because a mirrored digit does not keep its label (Part 5).

Lab 1 debrief: read the curves and the errors



[10 min, 13:30] Collect evidence from two or three pairs rather than a leaderboard. Ask each pair for its validation curves and one failure case with a hypothesis.

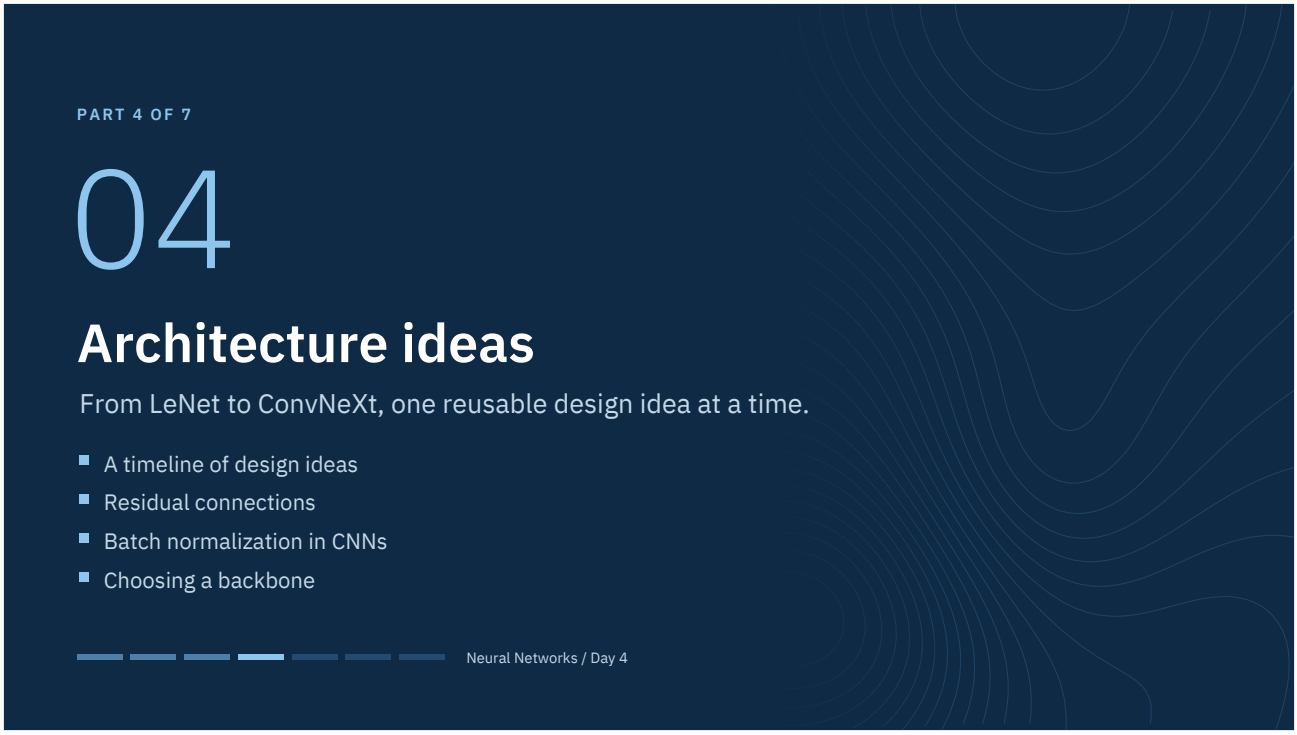
Shapes: if anyone changed the number of pooling layers, fc1's input size had to change too; that coupling is the most common Lab 1 error, and GAP removes it.

Curves: a training accuracy that keeps rising while validation accuracy flattens or falls indicates overfitting, typically in the large dense head. In the solution run, SimpleCNN's validation accuracy peaks at about 68% after 5 to 8 epochs while its training accuracy climbs towards 99% and its validation loss rises; with BatchNorm and dropout the model reaches about 70% and is still improving at epoch 10. Differences between runs of a few tenths of a percent are within run-to-run noise on a single seed; do not over-read them.

Errors: ask which classes were confused and whether the images are ambiguous even to a person at 32 × 32 pixels. Resolution is part of the prediction contract.

Evidence: a comparison is only meaningful if one thing changed and both models were selected on validation data. Bridge to the afternoon: the architecture ideas that follow are each one "change" someone found worth keeping.

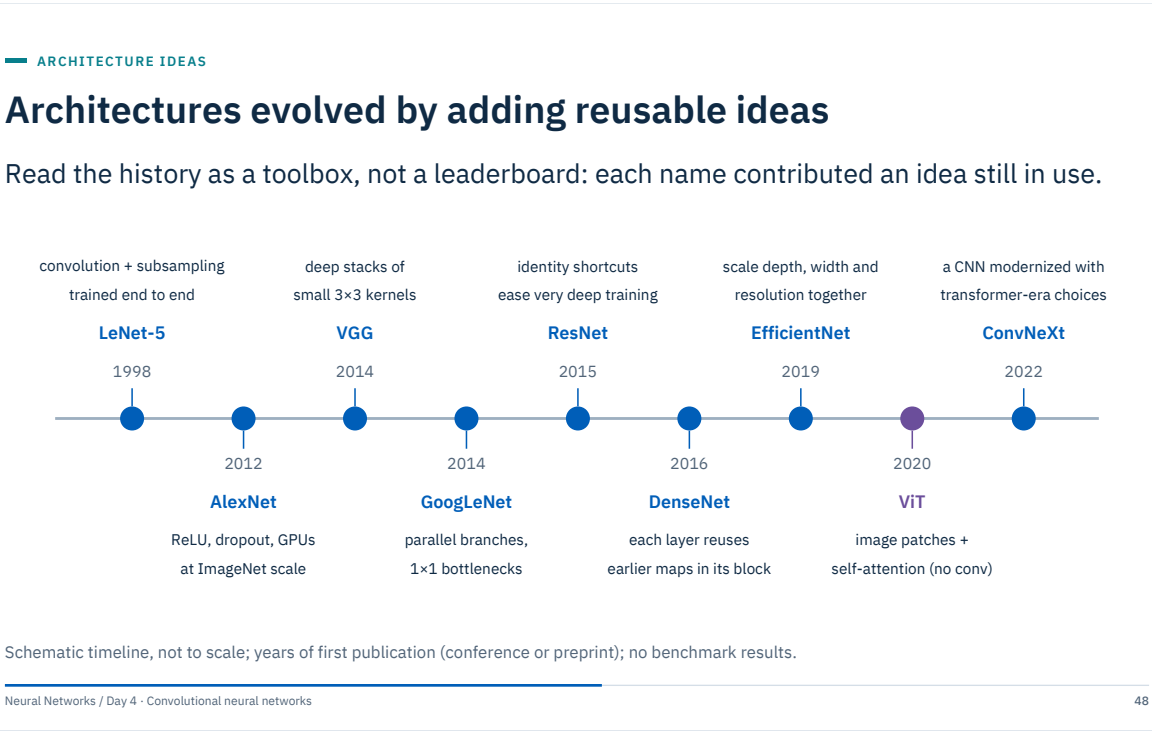
Architecture ideas



[About 30 minutes after the Lab 1 debrief.] The aim is not to memorize architectures but to recognize their building blocks: small stacked kernels, 1 × 1 bottlenecks, residual shortcuts, normalization, efficient scaling. Every modern backbone recombines these.

Keep the history qualitative. We deliberately show no accuracy numbers: benchmark results depend on data, preprocessing, training recipes and evaluation protocol, and they say little about performance on seismic, core or inspection images.

Architectures evolved by adding reusable ideas



[4 min] Walk left to right. LeNet-5 (LeCun et al., 1998) trained convolution and subsampling layers end to end for digit recognition. AlexNet (Krizhevsky, Sutskever and Hinton, 2012) combined ReLU, dropout, data augmentation and GPU training on ImageNet. VGG (Simonyan and Zisserman, 2014) stacked small 3 × 3 kernels, the argument from Part 2. GoogLeNet (Szegedy et al., 2014) ran parallel branches with 1 × 1 bottlenecks. ResNet (He et al., 2015) added identity shortcuts. DenseNet (Huang et al., 2016) fed every earlier map in a block to each later layer. EfficientNet (Tan and Le, 2019) scaled depth, width and resolution together. The Vision Transformer (Dosovitskiy et al., 2020) replaced convolution with self-attention over image patches. ConvNeXt (Liu et al., 2022) modernized a ResNet with design choices from the transformer era.

The spacing is not to scale, and the years are first appearances. ViT is drawn in purple because it is the one family without convolutions. The table on the next slide turns these into practical lessons.

Each architecture family leaves a practical lesson

ARCHITECTURE IDEAS

Each architecture family leaves a practical lesson

Family	Idea to retain	Practical lesson
LeNet / AlexNet	Learned convolutional feature hierarchies	Local features can support recognition
VGG	Repeated small kernels	Depth trades compute for capacity
Inception	Parallel branches, 1 × 1 bottlenecks	Mix scales at modest cost
ResNet / DenseNet	Shortcut paths between layers	Identity paths help optimization
MobileNet / EfficientNet	Efficiency-oriented design	Match capacity to deployment constraints

Neural Networks / Day 4 · Convolutional neural networks 49

[3 min] The table summarizes architectural ideas, not a leaderboard or a current performance ranking, so it quotes no accuracy figures: pretrained model performance depends on weights, preprocessing, dataset and evaluation protocol.

MobileNet (Howard et al., 2017) is the efficiency family not on the timeline: it factorizes a convolution into a depthwise step (one kernel per channel) and a 1 × 1 pointwise step, cutting computation for edge devices. Vision transformers and ConvNeXt are strong general-purpose backbones today; for small energy datasets the pretrained weights and the evaluation design usually matter more than the family.

Ask: for a drone that inspects solar farms and must run on board, which row matters most? The efficiency row, measured on the actual hardware.

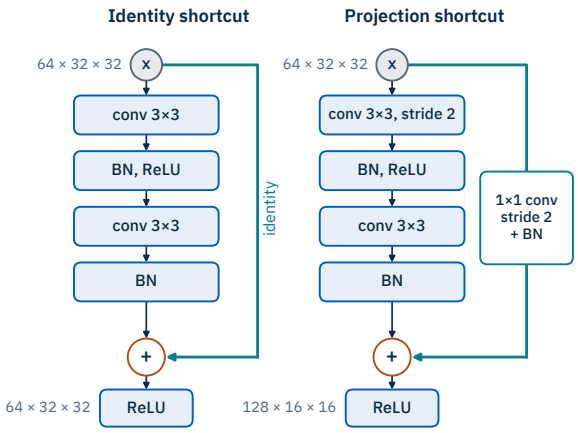
Residual connections preserve an identity path

ARCHITECTURE IDEAS

Residual connections preserve an identity path

$$y = x + F(x)$$

- The branch *F* learns a correction to its input.
- Addition needs matching shapes, or a 1 × 1 projection shortcut.
- The identity route helps gradients and signals pass through deep stacks.



Schematic: identity shortcut (left) and projection shortcut (right).

Neural Networks / Day 4 · Convolutional neural networks 50

[3 min] A residual block computes y = x + F(x): the stacked layers learn a correction F to the input rather than a whole new representation. If the best thing a block can do is nothing, it only has to drive F towards zero. During backpropagation the addition passes gradients straight through the identity path, so very deep stacks remain trainable.

Left: the identity version, where input and output have the same shape. Right: when the block halves resolution and doubles channels, x cannot be added directly; a 1 × 1 convolution with stride 2 (usually followed by BatchNorm) projects it to the new shape.

Residual connections do not eliminate every vanishing-gradient problem or guarantee a good solution. Ask learners what is required if the branch changes channel count or spatial resolution: exactly the projection on the right, which the next exercise sizes.

Make the residual addition shape-compatible

PRACTICE

4 min Pairs

Make the residual addition shape-compatible

Input x is 64 × 32 × 32. The branch starts with a 3 × 3 convolution, stride 2, padding 1, 128 filters.

1. What shape does the branch output $F(x)$ have?
2. What must the shortcut do before the addition?
3. How many weights does a 1 × 1, stride-2 projection from 64 to 128 channels have, without bias?

[4 min]

1. Spatial size: floor((32 + 2 − 3)/2) + 1 = 16, so F(x) is 128 × 16 × 16, assuming the rest of the branch keeps that shape (stride 1, padding 1).

2. The shortcut must halve the resolution and change 64 channels to 128, so that element-wise addition is defined. A 1 × 1 convolution with stride 2, usually followed by BatchNorm, does both. Zero-padding the extra channels is an alternative that He et al. (2016) also tested, but the projection is the common choice.

3. 128 × 64 × 1 × 1 = 8,192 weights. With a bias it would be 8,320, but the bias is usually omitted because the following BatchNorm has its own shift.

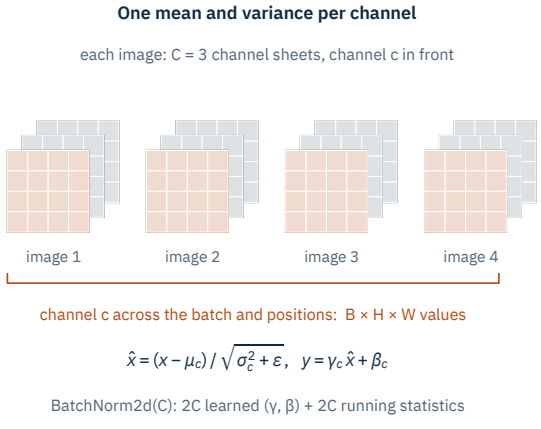
Common errors: adding tensors of different shapes and relying on broadcasting (PyTorch will raise an error or, worse, broadcast silently if a dimension is 1); forgetting the stride on the shortcut.

Batch normalization standardizes each channel

ARCHITECTURE IDEAS

Batch normalization standardizes each channel

- BatchNorm2d computes one mean and variance per channel over B × H × W.
- Learned scale γ and shift β restore flexibility.
- Evaluation uses running statistics, so train and eval modes differ.
- Tiny batches give noisy statistics; GroupNorm is one alternative.



Schematic of BatchNorm2d statistics; normalization in general is covered on Day 2.

Neural Networks / Day 4 · Convolutional neural networks 52

[3 min] Day 2 introduced normalization; here is what BatchNorm2d does inside a CNN. For each channel it pools all values of that channel across the batch and all spatial positions, B × H × W numbers, computes their mean and variance, and standardizes them. It then applies a learned per-channel scale γ and shift β. A BatchNorm2d over C channels has 2C learned parameters and keeps 2C running statistics (mean and variance), updated as moving averages during training.

In evaluation mode it uses the running statistics instead of the current batch, so a single image gives a deterministic output. This mode switch is the source of many transfer-learning surprises, next slide.

With very small batches (common for large seismic or casing images on limited GPU memory) the batch statistics are noisy; GroupNorm, which normalizes groups of channels within each sample, does not depend on batch size.

Frozen weights do not freeze BatchNorm statistics

ARCHITECTURE IDEAS

Frozen weights do not freeze BatchNorm statistics

```python
import torch
from torch import nn

torch.manual_seed(0)
bn = nn.BatchNorm2d(3)
for p in bn.parameters():
    p.requires_grad = False        # gamma and beta frozen

x = torch.randn(16, 3, 8, 8) * 2 + 5
bn.train(); bn(x)                  # training mode: statistics update
print(bn.running_mean)
bn.eval()                          # evaluation mode: running statistics
print(f"{bn(x).mean():.2f}")
```

OUTPUT

```
tensor([0.5045, 0.4930, 0.5014])
3.94
```

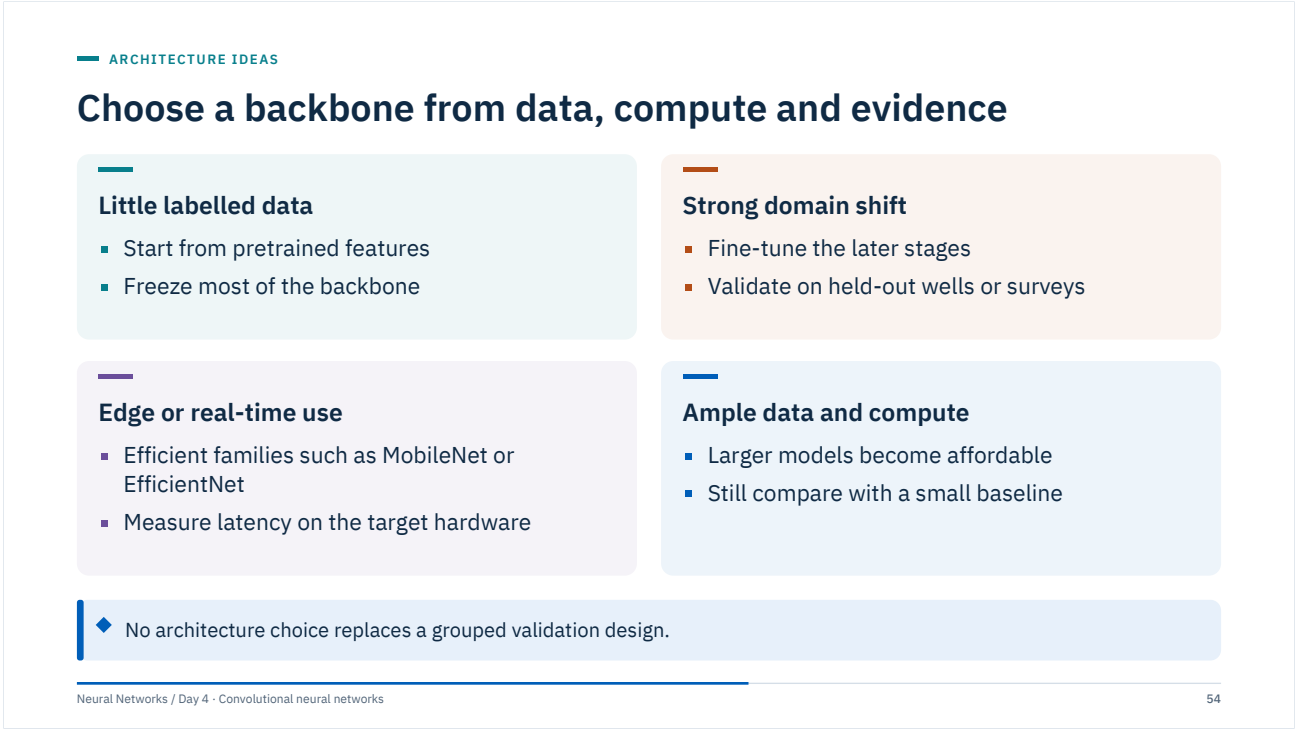
Neural Networks / Day 4 · Convolutional neural networks 53

[3 min] γ and β are frozen, yet one forward pass in training mode moved the running means from 0 towards the batch means of about 5: with PyTorch's default momentum of 0.1, one update moves them 10% of the way, to about 0.5. Running statistics are buffers, not parameters, so requires_grad does not protect them.

In evaluation mode the layer then normalizes with these barely updated statistics, and the output mean is 3.94 instead of roughly 0: the data are not normalized at all.

Two practical rules for transfer learning: during head-only training, keep the frozen backbone in evaluation mode (call backbone.eval() after model.train()), or its BatchNorm statistics drift towards your small, possibly unrepresentative batches; and always call model.eval() before validation and inference. This answers one of today's exit-check questions.

Choose a backbone from data, compute and evidence



[3 min] Offer these as starting points, not rules. With a few hundred labelled images, a pretrained backbone with a small trainable head is usually the most data-efficient start. When the imagery looks very different from the pretraining data (seismic amplitudes, ultrasonic thickness), expect to fine-tune more of the network, and validate on held-out wells or surveys, because that is where domain shift shows. For on-board inspection, latency and memory on the actual device decide. With large labelled archives, larger models become affordable, but they still need to beat a small, well-tuned baseline on the same grouped split.

Ask the room for their own case and let them place it. Then close the part with a short matching exercise.

Match each design idea to the problem it solves

PRACTICE

3 min · Individual

Match each design idea to the problem it solves

Choose from: residual connection, global average pooling, 1 × 1 bottleneck, batch normalization.

1. A deep plain stack trains worse than a shallower one.
2. A flatten-and-dense head holds 99% of the parameters.
3. 256 channels make every 3 × 3 layer expensive.
4. Activation scales drift between layers during training.

[3 min]

1. Residual connections. A deeper plain network training worse than a shallower one, even on the training set, is the degradation problem that motivated ResNet; identity shortcuts make it easy for extra layers to do no harm.

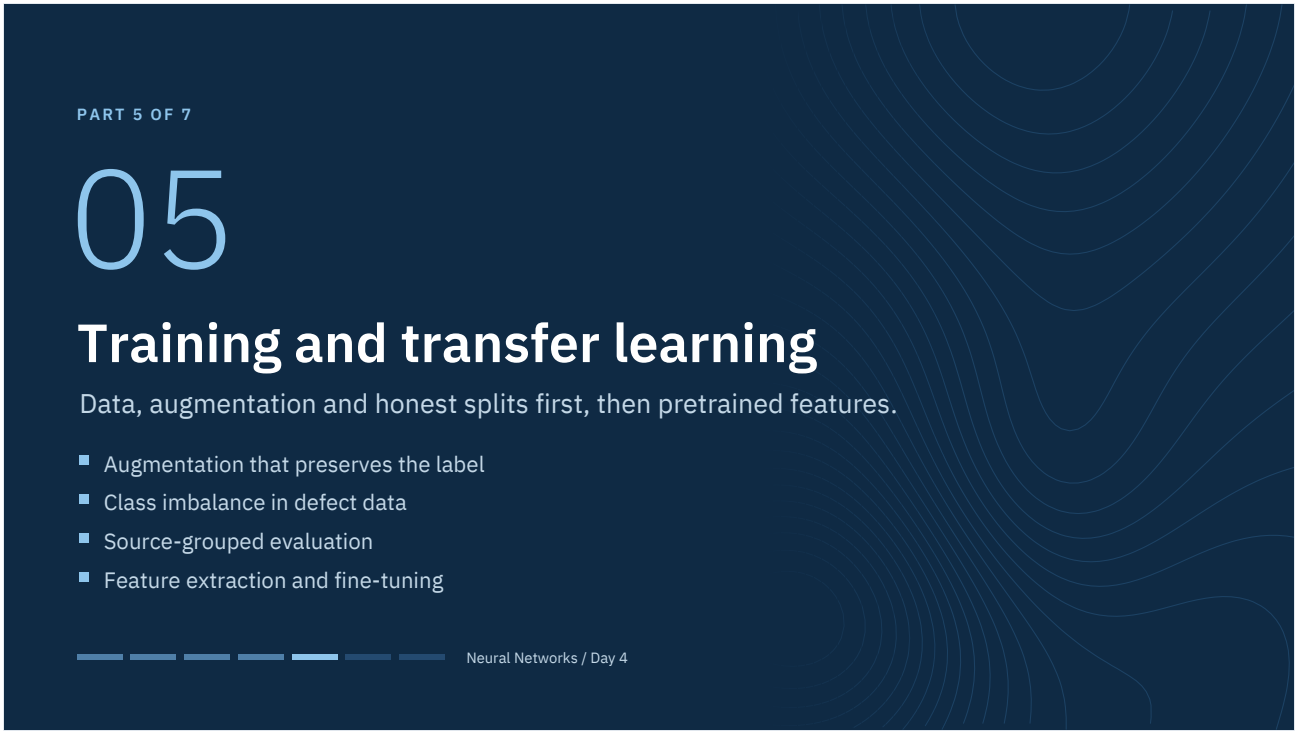
2. Global average pooling. For SimpleCNN, whose last convolution has 64 channels, a GAP head is one Linear(64, 10) layer with 650 parameters instead of the 2.1-million-parameter flatten head; the compact CNN's GAP head has 330.

3. A 1 × 1 bottleneck: reduce 256 channels to, say, 64 before the 3 × 3 layer and expand afterwards. The 3 × 3 layer then costs 64 × 64 × 9 instead of 256 × 256 × 9 weights, a factor of 16 fewer.

4. Batch normalization (or another normalization layer), which standardizes each channel's activations.

Accept reasonable alternatives with justification: for problem 1, careful initialization and normalization also help, but shortcuts are the defining fix.

Training and transfer learning



[About 40 minutes.] Day 2 covered the general training toolkit: optimizers, schedules, regularization, early stopping. This part covers what is specific to images: which augmentations are valid, how rare defect classes distort training and metrics, why image patches need grouped splits, and how to reuse a pretrained backbone.

The through-line: every choice here is a claim about the data. An augmentation claims the label is unchanged; a split claims the test data are independent of training; a pretrained backbone claims that features learned on other images transfer. Each claim can be checked, and the part ends with an exercise that asks learners to make those claims explicit.

Augmentation creates plausible new training views

TRAINING AND TRANSFER LEARNING

Augmentation creates plausible new training views

Each transform encodes a claim: the label does not depend on this change.



scikit-learn sample image china.jpg (CC BY 2.0), 360 × 360 crop resized to 160 × 160; transforms computed with NumPy and SciPy, seed 3.

Neural Networks / Day 4 · Convolutional neural networks 57

[3 min] Augmentation generates new training images by transforming existing ones: a mirror image, a random crop resized back, a small rotation, a brightness change, added sensor noise. The model sees more variation of the kind we expect at deployment and is discouraged from relying on exact pixel positions or exposure. Day 2 introduced augmentation as regularization; here the question is which transforms are valid.

Each image is still "a pagoda", so each transform preserves this label. Point to the rotation: it introduces black corners unless you crop them away or fill them, for example by reflection, and a model can learn that corners mean "augmented". Point to brightness: clipping at 255 destroys information in bright regions.

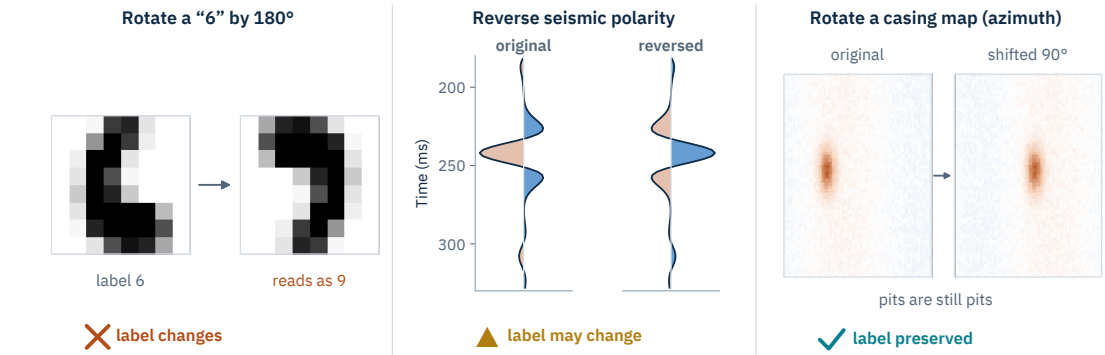
Augmentation is applied on the fly, independently every epoch, and only to training data. Validation and test images go through deterministic preprocessing. Transition: for many energy images, some of these transforms are not harmless.

Some transforms silently change the label

TRAINING AND TRANSFER LEARNING

Some transforms silently change the label

Whether a transform is safe depends on the physics and on how the target is defined.



scikit-learn digit; synthetic seismic trace (seed 7); synthetic thickness map (seed 5) shifted by 18 of 72 sectors.

Neural Networks / Day 4 · Convolutional neural networks 58

[4 min] Left: rotating a 6 by 180° produces something that reads as a 9. The transform is fine for photographs of pagodas and wrong for digits. A horizontal flip does not preserve digit identity either, which is why the SVHN notebook's augmentation uses only small random crops and colour jitter; ask learners why it leaves out the flip.

Middle: reversing seismic polarity multiplies the trace by −1, turning troughs into peaks. If a label depends on the sign of a reflection, for example a soft event that marks a lower-impedance layer, a polarity flip changes its meaning. For a fault-presence label it may be harmless. Polarity conventions also differ between datasets, so check which one applies.

Right: shifting a casing map around the azimuth axis moves the pits but they remain pits, so a defect-presence label is preserved. It would not be if the target were orientation-specific, such as wear on the low side of a deviated well.

Rule: write down the label definition first, then allow only the transforms that provably keep it.

Augmentation must preserve the task label

PRACTICE

4 min Pairs

Augmentation must preserve the task label

You classify casing defects from inspection images.

1. Is a horizontal flip physically meaningful for this acquisition geometry?
2. Could cropping remove the defect and leave its original label?
3. Which transforms belong only in training?

Neural Networks / Day 4 · Convolutional neural networks 59

[4 min in pairs, 2 min discussion] Answers depend on acquisition geometry and target definition. Do not prescribe transformations without checking.

1. Flipping the azimuth axis mirrors the pipe, which is equivalent to viewing it from the other direction. For a defect-presence label that is plausible; for targets tied to orientation (high side, low side, tool-face-referenced features) it is not. Flipping depth reverses the order of joints and collars; it may be acceptable for local pits, but check how labels were defined.

2. Yes. A random crop from a "defective" patch can cut out the pit entirely, and the crop still carries the label "defective": a noisy label created by augmentation. Crop only around annotated extents, or recompute the label from the mask after cropping.

3. All random transforms (crops, flips, noise, brightness) belong only in training. Validation and test use fixed preprocessing. Split by source asset or original image before creating augmented samples, so related views cannot leak into validation.

Common error: augmenting the whole dataset once, saving the files, and then splitting randomly, which puts augmented copies of the same image on both sides.

Training and evaluation use different transforms

TRAINING AND TRANSFER LEARNING

Training and evaluation use different transforms

```python
import torch
from torchvision.transforms import v2

MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
train_tf = v2.Compose([
    v2.ToImage(),
    v2.RandomResizedCrop(224, scale=(0.8, 1.0), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ColorJitter(brightness=0.2),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(MEAN, STD),
])
eval_tf = v2.Compose([
    v2.ToImage(), v2.Resize(256, antialias=True), v2.CenterCrop(224),
    v2.ToDtype(torch.float32, scale=True), v2.Normalize(MEAN, STD),
])
```

ImageNet statistics shown; use those of your pretrained weights or of your training data.

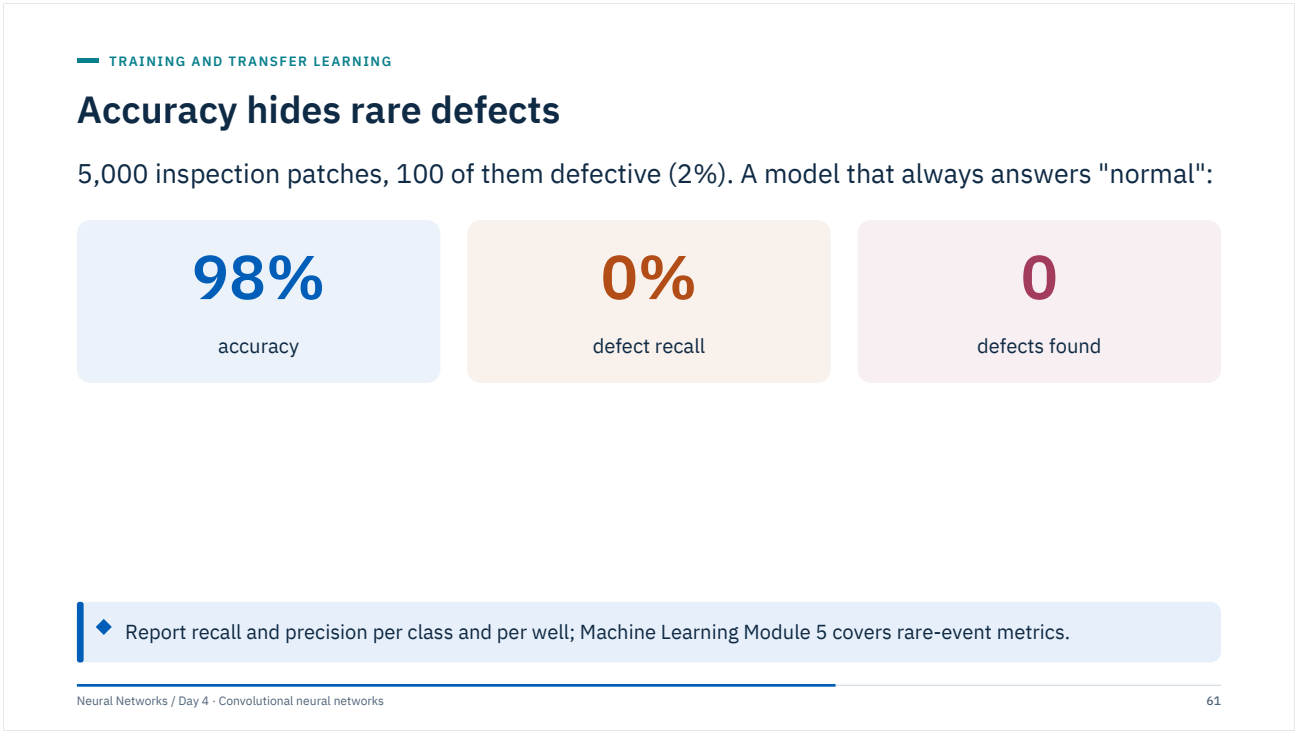
[3 min] This uses torchvision's transforms v2 API. The training pipeline converts to a tensor image, takes a random crop covering 80–100% of the area and resizes it to 224 × 224, flips half the images left to right, jitters brightness, converts to floats in [0, 1] and normalizes each channel. The evaluation pipeline is deterministic: resize the short side to 256, take the central 224 × 224 crop, convert and normalize with the same statistics.

The mean and standard deviation are the per-channel ImageNet statistics commonly used with ImageNet-pretrained weights. For a model trained from scratch on your own data, compute them on the training split only; computing them on all data leaks information. The Lab 2B notebook computes its statistics on the training wells only.

For single-channel energy images (thickness, amplitude), these RGB statistics do not apply. Whether a flip or colour jitter is valid is the question from the previous exercise, not a default.

Reference: https://docs.pytorch.org/vision/stable/transforms.html

Accuracy hides rare defects



[2 min] This is a constructed illustration, not a dataset statistic: 4,900 normal and 100 defective patches. A model that never predicts a defect is 98% accurate and completely useless: recall on the defect class is 0 of 100.

Defect imagery is almost always imbalanced, and the imbalance also changes training: with plain cross-entropy the easiest way to lower the loss is to predict "normal" confidently. Two consequences follow. First, report per-class recall and precision, a confusion matrix, and results per well, not overall accuracy. Second, consider rebalancing the loss or the sampling, next slide.

The Machine Learning course, Module 5, covers precision-recall curves, event-level metrics and threshold choice in depth; cross-reference rather than repeat it.

Class weights rebalance the loss, not the data

TRAINING AND TRANSFER LEARNING

Class weights rebalance the loss, not the data

```
import torch
from torch import nn

y_train = torch.tensor([0] * 4900 + [1] * 100)
counts = torch.bincount(y_train, minlength=2)
weights = counts.sum() / (2 * counts.float())
print(weights)
criterion = nn.CrossEntropyLoss(weight=weights)
```

OUTPUT

```
tensor([ 0.5102, 25.0000])
```

- Each defective patch now weighs 49 times a normal one in the loss.
- Alternatives: WeightedRandomSampler, focal loss, threshold tuning.
- Choose the decision threshold on validation data.

Neural Networks / Day 4 · Convolutional neural networks 62

[3 min] The weights follow the "balanced" rule, n_samples / (n_classes × count): 5,000/(2 × 4,900) = 0.5102 for normal and 5,000/(2 × 100) = 25 for defective. Their ratio is 25/0.5102 = 49, the inverse of the class ratio 4,900/100. Compute the counts on the training split only.

Alternatives: WeightedRandomSampler draws defective patches more often without changing the loss; focal loss down-weights easy examples; undersampling the majority (the Lab 2B notebook keeps at most 2 normal patches per defective one, in the training wells only, after the split) discards data and, if done before splitting, also distorts the test prevalence.

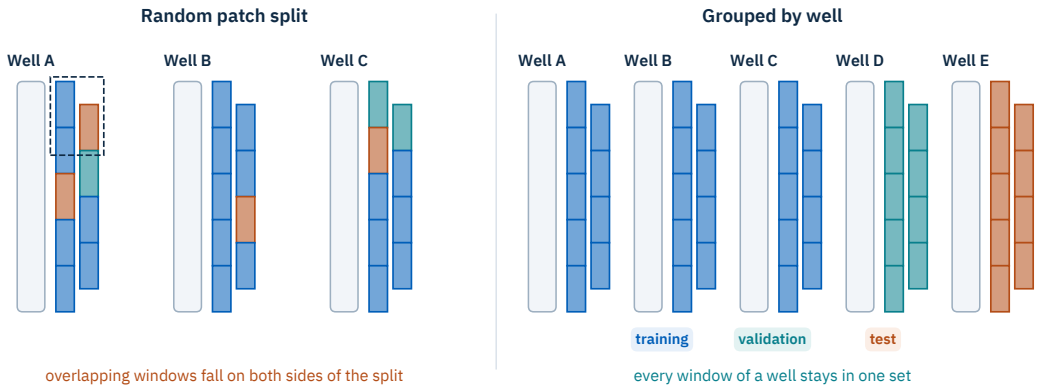
Weighting shifts the model's probabilities, so the default 0.5 threshold is no longer meaningful. Choose the threshold on validation data according to the cost of a missed defect versus a false alarm, and report precision and recall at that threshold on the untouched test set, at its natural prevalence.

A valid image evaluation separates sources

TRAINING AND TRANSFER LEARNING

A valid image evaluation separates sources

Overlapping windows are near-duplicates: split by well, survey or asset first.



Schematic: 32-row windows with stride 16 (50% overlap); random assignment drawn with seed 8.

[3 min] Left: windows cut from a long log with 50% overlap, then assigned randomly. The dashed box marks a training window and a test window that share half their rows. The model is tested on data it has partly seen, and patches from the same well share tool settings, casing grade and noise, so even non-overlapping patches are more alike than patches from a new well.

Group related crops or patches from the same original image. Hold out assets, surveys or acquisition batches when deployment requires transfer. Keep validation transforms deterministic. Inspect mistakes by class, source and image quality.

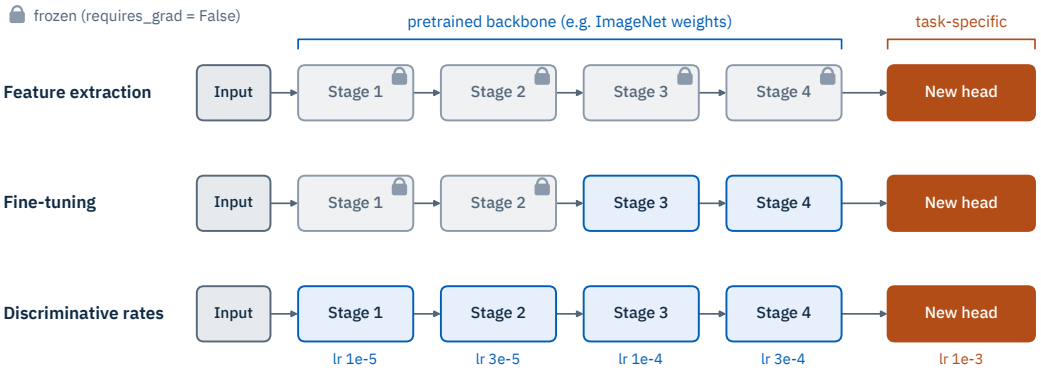
Random patch splits can create near-duplicate training and test images. A high patch accuracy does not prove useful defect-level recall. Define how patch predictions become an asset-level or defect-level decision. In scikit-learn, GroupShuffleSplit or GroupKFold with groups set to the well identifier does this; the Machine Learning course, Module 1, covers grouped splits in general.

Transfer learning reuses a pretrained backbone

TRAINING AND TRANSFER LEARNING

Transfer learning reuses a pretrained backbone

Pretrained features are a hypothesis about the new domain; test it.



Schematic of three adaptation strategies; learning rates are illustrative.

Neural Networks / Day 4 · Convolutional neural networks 64

[3 min] A backbone pretrained on a large natural-image dataset has learned general filters (edges, textures, parts) in its early stages and more task-specific features in later stages. Replace its classification head with a new one for your classes.

Top: feature extraction freezes the whole backbone and trains only the head; cheap, and hard to overfit with a few hundred images. Middle: fine-tuning also unfreezes the later stages, letting them adapt to the new domain. Bottom: discriminative learning rates train everything, with smaller rates near the input so that generic early features change slowly.

Domain shift can be substantial between natural images and seismic, core or acoustic imagery, so transfer must be measured. Pretrained RGB models also expect three channels: for single-channel data, repeat the channel three times or adapt the first convolution.

Adaptation proceeds from frozen features to fine-tuning

TRAINING AND TRANSFER LEARNING

Adaptation proceeds from frozen features to fine-tuning

Stage	Trainable parameters	Main question
Feature extraction	New task head	Are the existing features useful?
Fine-tuning	Head and later backbone stages	Does adaptation improve validation performance?
Discriminative rates	All, smaller rates near the input	Does gentler updating of early layers help?

Use the preprocessing that belongs to the chosen pretrained weights.

[2 min] Start with a baseline, then unfreeze selectively with a smaller learning rate where appropriate. Each row answers a question with evidence from validation data, not with a default. If feature extraction already matches fine-tuning on grouped validation data, prefer the simpler model.

BatchNorm running statistics can change even when weight gradients are frozen, so define train/eval behaviour deliberately during head-only training, as the BatchNorm slide showed.

The last line matters more than it looks. Pretrained weights were trained with specific resizing, cropping and normalization; feeding differently scaled inputs silently degrades the features. torchvision attaches the matching transform to each weights object, shown next.

Load pretrained weights with their own preprocessing

TRAINING AND TRANSFER LEARNING

Load pretrained weights with their own preprocessing

```python
from torchvision.models import resnet18, ResNet18_Weights
from torch import nn

weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
for parameter in model.parameters():
    parameter.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, n_classes)
preprocess = weights.transforms()
```

The new `fc` layer is created after freezing, so only its parameters train.

Neural Networks / Day 4 · Convolutional neural networks 66

[2 min] This may download pretrained weights, so prepare them before class if network access is limited. ResNet18_Weights.DEFAULT selects the best available weights for this architecture; weights.transforms() returns the matching preprocessing, including resize, crop and normalization.

Freezing sets requires_grad = False on every existing parameter. The replacement fc layer is created afterwards, so its parameters default to requires_grad = True: only the head trains. Pass only trainable parameters to the optimizer, for example filter(lambda p: p.requires_grad, model.parameters()).

Train only parameters with requires_grad = True. Set the backbone to evaluation behaviour for a fixed-feature baseline, then manage modes explicitly when fine-tuning. n_classes is your number of classes.

Reference: https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html ; https://docs.pytorch.org/vision/stable/models.html

Discriminative learning rates use parameter groups

TRAINING AND TRANSFER LEARNING

Discriminative learning rates use parameter groups

```
import torch

for name, p in model.named_parameters():
    p.requires_grad = name.startswith(("layer4.", "fc."))

optimizer = torch.optim.AdamW([
    {"params": model.layer4.parameters(), "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3},
], weight_decay=1e-4)
```

Frozen stages stay out of the optimizer; keep their BatchNorm layers in eval mode while fine-tuning.

[2 min] Continuing with the ResNet-18 from the previous slide, whose last stage is called layer4 and whose head is fc. The loop unfreezes only those two; str.startswith accepts a tuple of prefixes. The optimizer then receives two parameter groups with their own learning rates: the pretrained last stage moves ten times more slowly than the new head.

Extending this to all stages with gradually smaller rates towards the input gives the bottom row of the transfer diagram. The rates here are common starting points, not recommendations; tune them on validation data.

Common errors: passing model.parameters() to the optimizer, which includes frozen parameters (harmless but wasteful, and confusing when you later unfreeze); unfreezing without lowering the rate, which can wipe out pretrained features in the first epochs; forgetting that model.train() also switches the frozen stages' BatchNorm layers back to training mode.

Choose an adaptation strategy for three datasets

PRACTICE

5 min · Pairs

Choose an adaptation strategy for three datasets

For each dataset: feature extraction, fine-tuning or training from scratch? And how would you split it?

1. 2,000 labelled core photographs of six lithologies from 12 wells.
2. 300 labelled seismic patches from one survey, to be used on a second survey.
3. 50,000 labelled ultrasonic casing patches from 40 wells.

Neural Networks / Day 4 · Convolutional neural networks 68

[5 min in pairs, 3 min discussion] Accept any answer that is justified and paired with a sensible split.

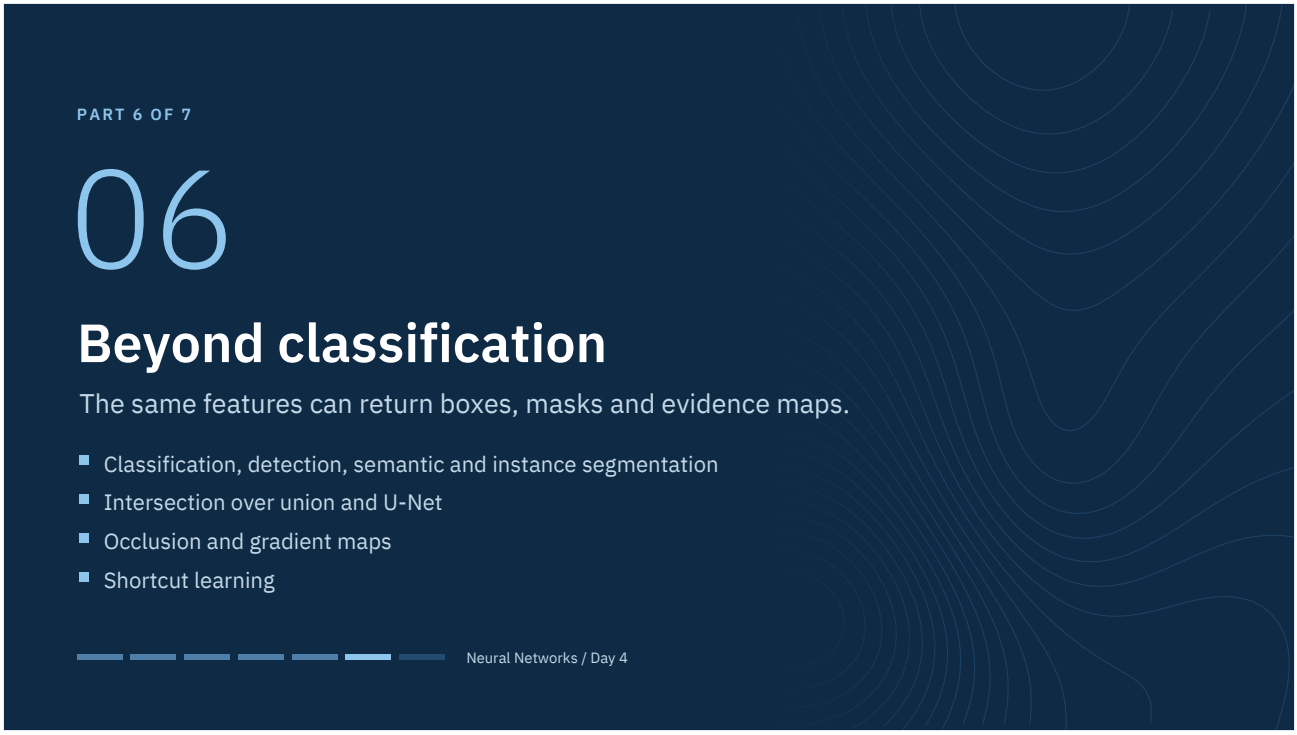
1. Core photographs are RGB images reasonably close to natural photographs. Start with feature extraction from an ImageNet-pretrained backbone, then try fine-tuning later stages. Split by well (grouped cross-validation over 12 wells, or hold out 2 to 3 wells), never by random photograph, because adjacent core boxes from one well look alike.

2. 300 patches is very little, and the target is a different survey: the real question is transfer across surveys. Use a pretrained backbone with feature extraction or light fine-tuning, strong physically valid augmentation, and above all a validation set from the second survey, even a small labelled one. Report the within-survey score only as a reference.

3. 50,000 patches from 40 wells is enough to fine-tune fully or even train a compact CNN from scratch, since single-channel thickness images differ from photographs. Split by well (for example 28/6/6 wells), keep the natural defect prevalence in validation and test, and report per-well recall.

Common error: choosing the strategy by dataset size alone and ignoring where the model will be used.

Beyond classification



[About 25 minutes.] So far every model returned one label per image. Many energy questions need more: where is the defect, how large is it, which pixels belong to the fault. This part shows how the same convolutional features feed different heads, how detections are scored, how an encoder-decoder produces masks, and how to check what a model actually relies on.

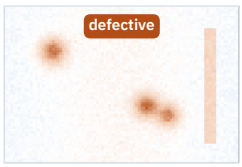
Keep detection and instance segmentation brief: the aim is to recognize the output formats and their evaluation, not to train a detector today.

Vision tasks differ in what they return

BEYOND CLASSIFICATION

Vision tasks differ in what they return

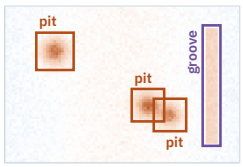
The label format decides the model, the loss and the metric.



Classification

one label per image

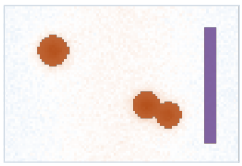
“defects present”



Object detection

box + class + score per object

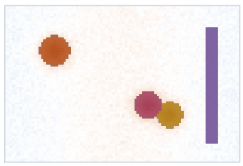
matched to truth by IoU



Semantic segmentation

a class for every pixel

pits merge into one region



Instance segmentation

a separate mask per object

touching pits stay separate

Synthetic thickness patch (seed 12) with three pits and a groove; labels derived from the known geometry, not from a model.

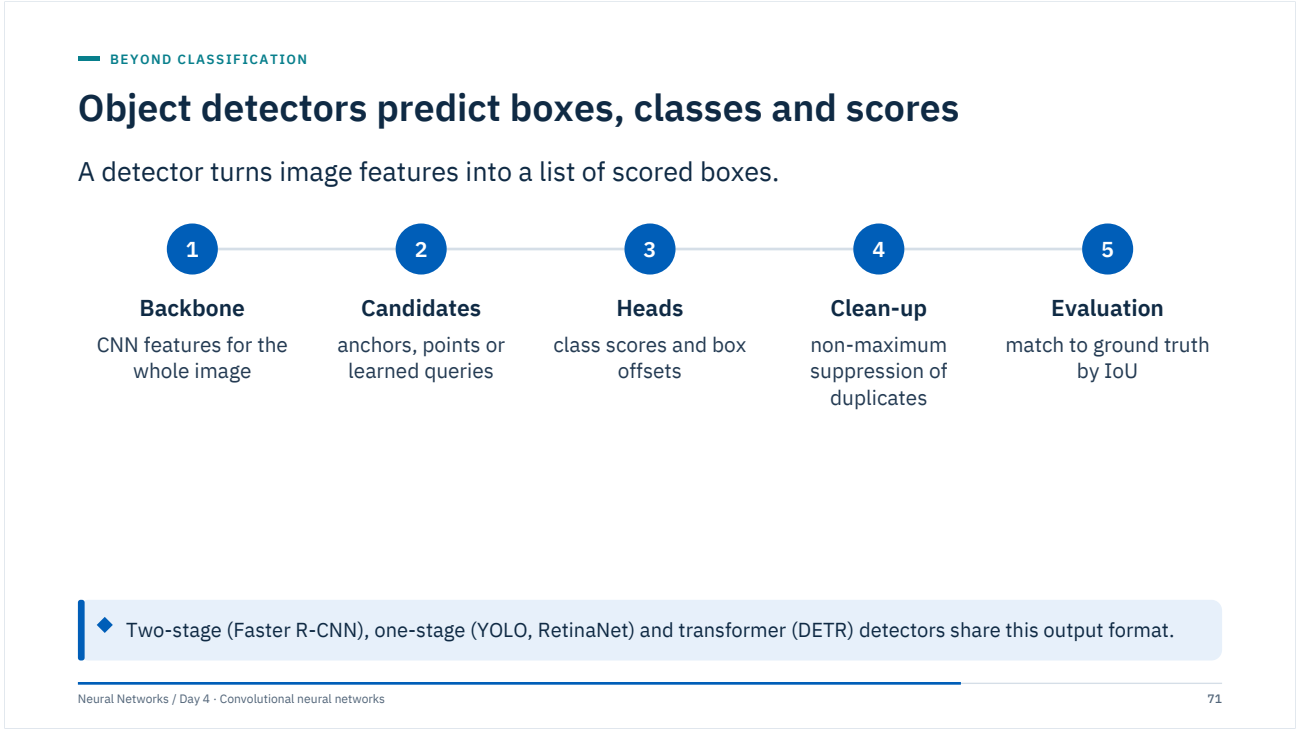
Neural Networks / Day 4 · Convolutional neural networks 70

[3 min] One synthetic casing patch, four label formats. Classification returns one label per image: cheap to annotate, but it does not say where. Object detection returns a box, a class and a confidence score per object; boxes are matched to ground truth by intersection over union. Semantic segmentation assigns a class to every pixel; note that the two touching pits merge into one orange region, because semantic masks do not separate objects of the same class. Instance segmentation gives each object its own mask, so the touching pits stay separate and can be counted and measured individually.

Annotation cost rises from left to right: a label per image, a box per object, then outlines per pixel. Ask: which format would an integrity engineer need to estimate remaining wall thickness over a pit? A mask, because extent and depth matter, not just presence.

These label formats are drawn from the known synthetic geometry; no model produced them.

Object detectors predict boxes, classes and scores



[3 min] Walk the stages. A backbone, often pretrained, produces feature maps. The detector proposes many candidate locations: predefined anchor boxes, points on the feature grid, or learned object queries. For each candidate, heads predict class scores and offsets that adjust the box. Many candidates fire on the same object, so non-maximum suppression keeps the highest-scoring box and removes overlapping duplicates; DETR-style detectors avoid this step by matching predictions to objects one to one during training. Finally, predictions are matched to ground-truth boxes by IoU to count true and false positives.

Two-stage detectors first propose regions and then classify them; one-stage detectors predict directly on the grid. For the course, the important parts are the output contract (box, class, score) and the evaluation, next slide.

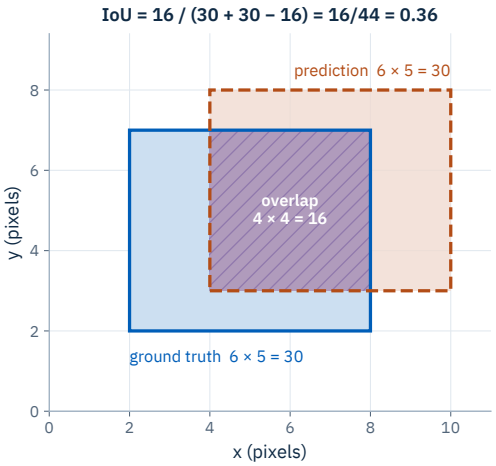
Intersection over union scores box overlap

BEYOND CLASSIFICATION

Intersection over union scores box overlap

$$\mathrm{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

- Here the overlap is 16 and the union 44: IoU = 0.36.
- A detection counts only above a threshold, often 0.5.
- COCO-style scores average over thresholds 0.50 to 0.95.



Exact intersection over union for two axis-aligned boxes.

Neural Networks / Day 4 · Convolutional neural networks 72

[3 min] Work the numbers. Ground truth spans x from 2 to 8 and y from 2 to 7: 6 × 5 = 30. The prediction spans 4 to 10 and 3 to 8: also 30. The overlap spans x from 4 to 8 and y from 3 to 7: 4 × 4 = 16. The union is 30 + 30 − 16 = 44, so IoU = 16/44 = 0.36.

IoU ranges from 0 (no overlap) to 1 (identical boxes) and does not depend on image size. With the common threshold of 0.5 (the PASCAL VOC convention), this prediction would be a false positive, even though it clearly found the object: a reminder that the threshold is part of the metric definition. COCO-style average precision averages over IoU thresholds from 0.50 to 0.95 in steps of 0.05, rewarding precise localization.

For thin or small targets such as pits, IoU is very sensitive to a few pixels of offset; agree on the threshold with the people who use the boxes.

Decide whether each detection is a true positive

PRACTICE

4 min Pairs

Decide whether each detection is a true positive

Ground-truth box corners: (0, 0) and (4, 4). A detection counts if IoU ≥ 0.5.

1. Detection A has corners (1, 1) and (5, 5). Compute its IoU.
2. Detection B has corners (0, 1) and (4, 5). Compute its IoU.
3. Both are predicted for the same defect. How many true and false positives?

Neural Networks / Day 4 · Convolutional neural networks 73

[4 min]

A: overlap x from 1 to 4 and y from 1 to 4, so 3 × 3 = 9. Areas 16 and 16. Union 16 + 16 − 9 = 23. IoU = 9/23 = 0.39: below 0.5, so a false positive.

B: overlap x from 0 to 4 and y from 1 to 4, so 4 × 3 = 12. Union 16 + 16 − 12 = 20. IoU = 12/20 = 0.60: a true positive.

Together: one true positive (B) and one false positive (A). Even if A had also exceeded the threshold, only one detection may match each ground-truth object; the second would count as a duplicate false positive, which is exactly what non-maximum suppression tries to prevent.

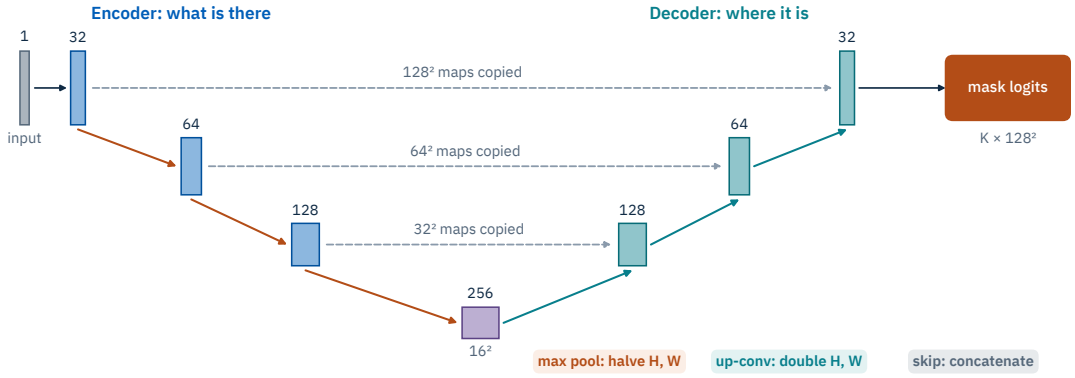
Common errors: dividing the overlap by one box's area instead of the union; subtracting corners incorrectly when boxes do not share an edge.

U-Net segments with an encoder, decoder and skips

BEYOND CLASSIFICATION

U-Net segments with an encoder, decoder and skips

The encoder gathers context, the decoder restores resolution, and skips carry back fine detail.



Schematic of a small U-Net for a 128 × 128 input; numbers are channel counts. After Ronneberger et al. (2015).

Neural Networks / Day 4 · Convolutional neural networks 74

[3 min] Segmentation needs an output at the input resolution. The encoder is an ordinary CNN: convolutions and pooling halve the size at each level while channels grow, from 128 × 128 down to 16 × 16 here, so deep units see large context. The decoder reverses this with up-convolutions (transposed convolutions) that double the size at each level.

Upsampling alone would give blurry masks, because the fine detail was pooled away. U-Net's key idea is the skip connection: at each level, the encoder's feature maps are copied across and concatenated with the decoder's, so the decoder combines "what" from deep layers with "where" from shallow ones. A final 1 × 1 convolution gives K class logits per pixel, trained with a per-pixel cross-entropy or a Dice loss.

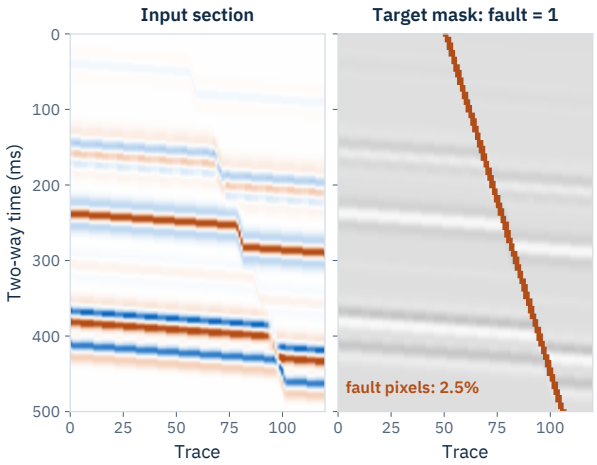
U-Net was introduced for biomedical images (Ronneberger, Fischer and Brox, 2015) and variants are widely used for seismic fault and salt segmentation and for defect masks.

Segmentation masks bring their own label problems

BEYOND CLASSIFICATION

Segmentation masks bring their own label problems

- Pixel labels are slow to draw and differ between interpreters.
- Targets are thin and rare: here 2.5% of pixels are fault.
- Score with Dice or IoU per class, with tolerance near boundaries.
- **Instance segmentation** (e.g. Mask R-CNN) also separates objects.



Synthetic section (seed 7); fault mask: pixels within 1.5 traces of the fault plane.

[3 min] Left: the synthetic section from Part 1. Right: a fault mask, a band three traces wide around the fault plane. Only 2.5% of pixels are fault, so a model predicting "no fault" everywhere scores 97.5% pixel accuracy: the same imbalance trap as before. Use per-class IoU or the Dice coefficient (2|A∩B| / (|A| + |B|)) for the fault class, and consider class-weighted or Dice losses.

Real fault labels are interpretations: two interpreters draw slightly different lines, and a one-trace offset should not count as a complete miss. Many studies therefore score with a distance tolerance around the boundary.

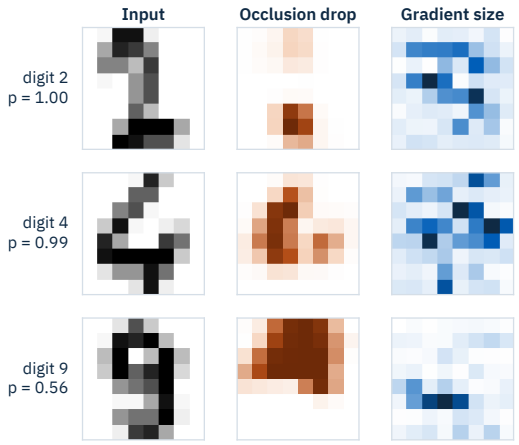
Instance segmentation (for example Mask R-CNN) predicts a separate mask per object, which matters when objects touch, as the pits did two slides ago. Synthetic data like this is useful for pretraining and tests, but the gap to field data must be measured, not assumed.

Occlusion maps show which pixels move the score

BEYOND CLASSIFICATION

Occlusion maps show which pixels move the score

- **Occlusion**: blank a patch, record the drop in the class probability.
- Gradient maps: how strongly each pixel changes the class logit.
- Both measure local sensitivity; neither explains the reasoning.



Tiny digits CNN (seed 0): 2 × 2 patches set to 0; absolute input gradients; each map scaled to its maximum.

Neural Networks / Day 4 · Convolutional neural networks 76

[3 min] Occlusion sensitivity slides a blank patch over the image and records how much the predicted probability of the true class drops; large drops mark pixels the prediction depends on. For the 2 and the 4, the drops sit on the strokes that distinguish them. The 9 is predicted with probability 0.56 only, and blanking almost any part of its loop lowers it.

Gradient maps compute the derivative of the class logit with respect to each input pixel: cheap, one backward pass, but noisy and sensitive to saturation.

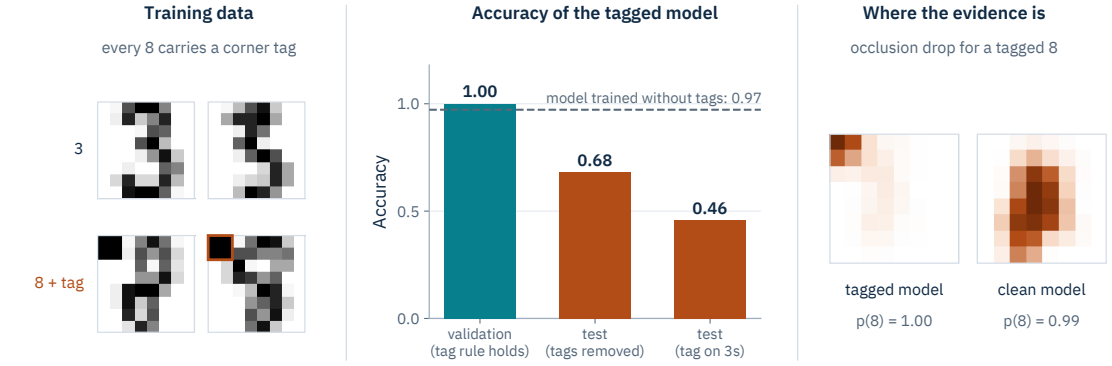
Neither method explains why the network combines evidence as it does, and both depend on choices (patch size, fill value, which output). Treat them as hypotheses to test: if the map highlights something physically meaningless, the next slide shows what can be going on.

Shortcuts can pass validation and fail in deployment

BEYOND CLASSIFICATION

Shortcuts can pass validation and fail in deployment

A bright tag on every training 8 is enough: validation is perfect, then accuracy collapses.



Tiny CNN (seed 0, CPU) on scikit-learn 3s and 8s: 214 training, 71 validation and 72 test images.

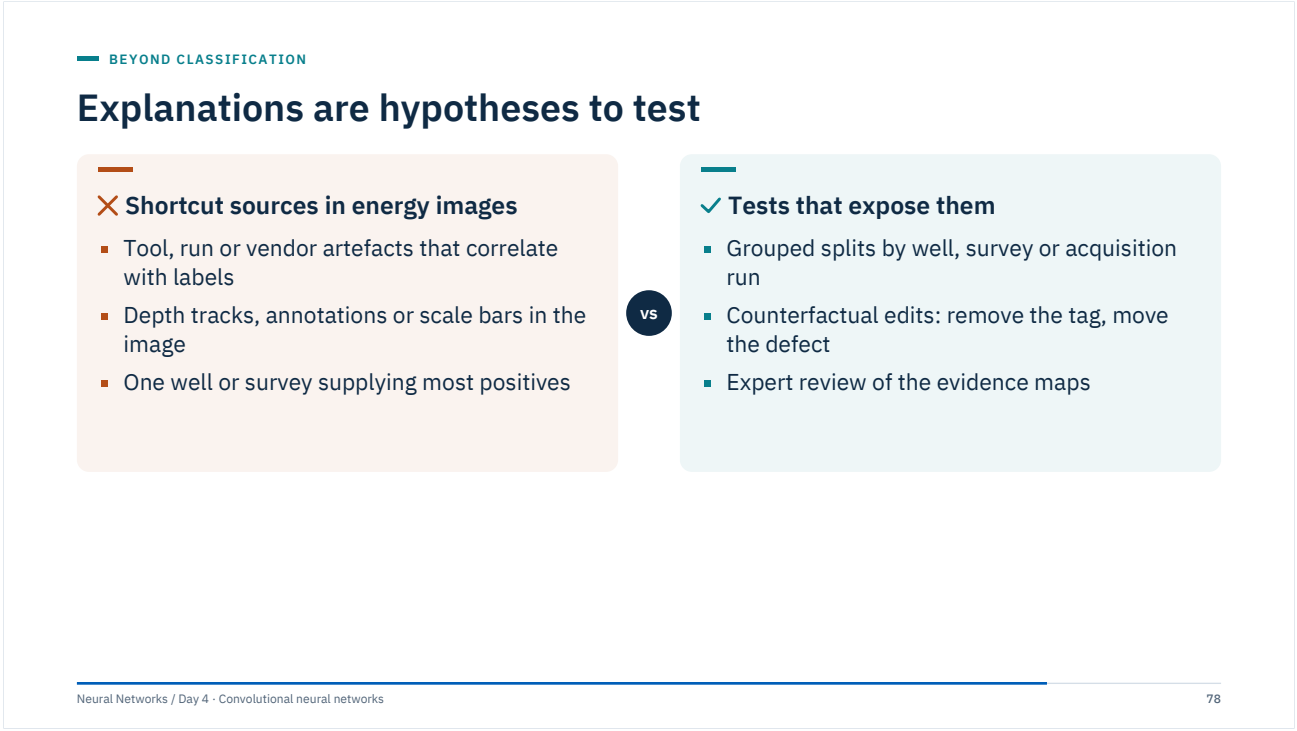
Neural Networks / Day 4 · Convolutional neural networks 77

[4 min] A controlled experiment. Every 8 in the training and validation sets carries a bright 2 × 2 tag in the corner; the 3s do not. The model reaches 1.00 validation accuracy, because the tag rule also holds there. On test images without tags, accuracy drops to 0.68 (49 of 72); with the tag moved onto the 3s it falls to 0.46 (33 of 72), worse than guessing. The same architecture trained without tags scores 0.97 on the clean test set, so the digits themselves were learnable.

The occlusion maps show why: for the tagged model, evidence for "8" sits in the corner; for the clean model, on the strokes.

This is shortcut learning (Geirhos et al., 2020): the model uses whatever feature separates the training labels most easily. Validation data drawn from the same process cannot detect it.

Explanations are hypotheses to test

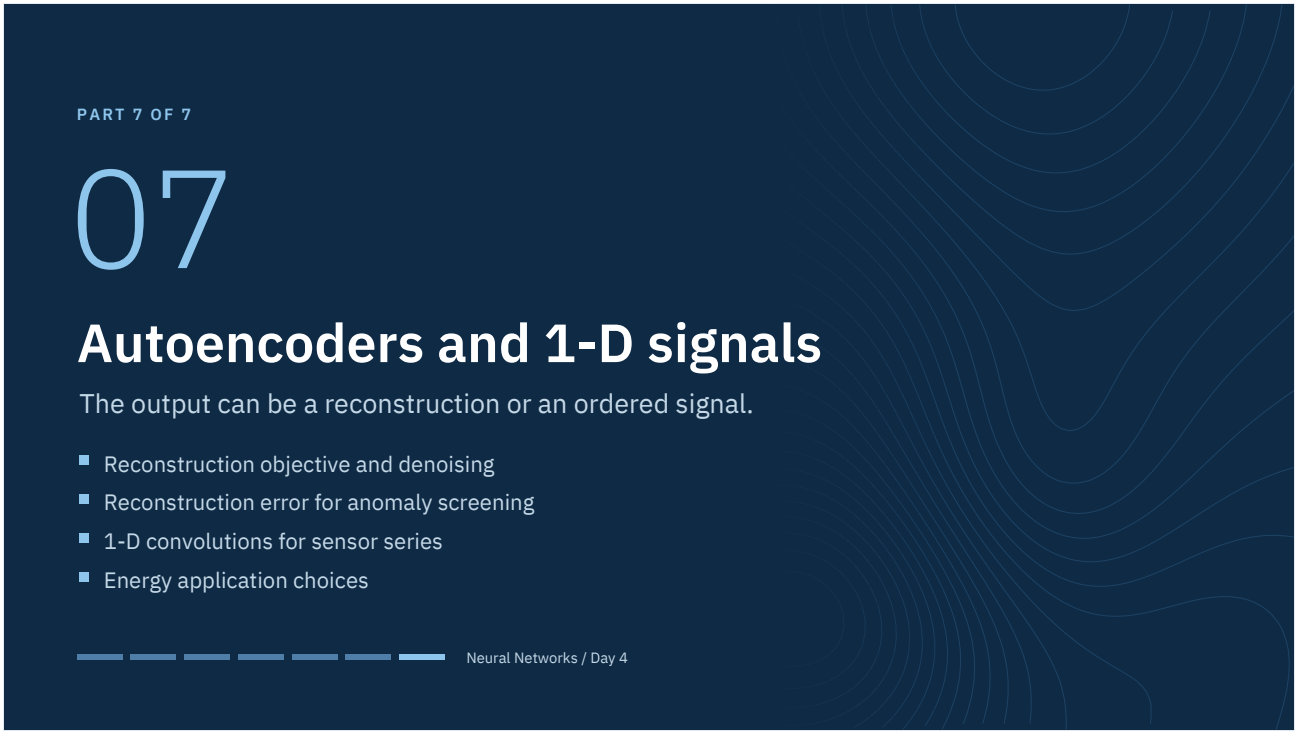


[3 min] Left: real versions of the corner tag. Defect examples collected from one logging run with a different gain setting; interpreted images with burned-in annotations or scale bars; positive examples concentrated in one well, survey or vendor format. Any feature that correlates with the label in the training data can become the model's evidence.

Right: tests that expose shortcuts. Grouped splits break correlations that are tied to a source. Counterfactual edits test a specific hypothesis: remove the annotation and see if the prediction changes; paste a synthetic pit into a clean image and see if it is detected. Domain experts reviewing occlusion maps can spot evidence in physically implausible places.

Ask the room for one shortcut they suspect in their own data. Transition: the last part changes the output again, to reconstructions and ordered signals.

Autoencoders and 1-D signals



[About 20 minutes.] Connect representation learning to the autoencoder introduction on Day 2 and, for learners who took the Machine Learning course, to its PCA compression example (Module 4). Autoencoders add nonlinear learned mappings but require evaluation of what is retained and what is lost.

The second half turns the same convolution machinery to one-dimensional ordered data, logs and sensor series, as a bridge to Day 5 on sequence models. Keep it brisk: Lab 2 starts at 15:50.

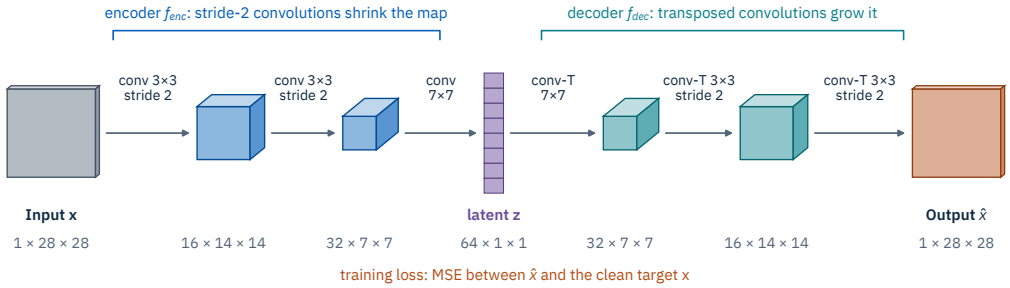
An autoencoder learns a reconstruction objective

AUTOENCODERS AND 1-D SIGNALS

An autoencoder learns a reconstruction objective

$$z = f_{enc}(x), \qquad \hat{x} = f_{dec}(z), \qquad L = \|x - \hat{x}\|^2$$

A bottleneck prevents copying; denoising maps corrupted inputs to clean targets.



Schematic of the Lab 2A autoencoder (latent size 64); shapes checked with PyTorch, batch dimension omitted.

Neural Networks / Day 4 · Convolutional neural networks 80

[3 min] An autoencoder has no labels: the encoder compresses the input x into a latent code z, the decoder reconstructs $\hat{x}$ from z, and training minimizes the reconstruction error. Here the encoder uses two stride-2 convolutions (28 → 14 → 7) and a 7 × 7 convolution down to a 64-dimensional code; the decoder mirrors it with transposed convolutions, which learn to upsample. This is exactly the Lab 2A model; it has 210,369 parameters.

A bottleneck or other constraint limits trivial copying: if z were as large as x, the network could learn the identity. Denoising trains against a clean target from a corrupted input, which forces the model to learn the structure of clean images rather than copying pixels. Convolutional layers retain spatial structure in the encoder and decoder.

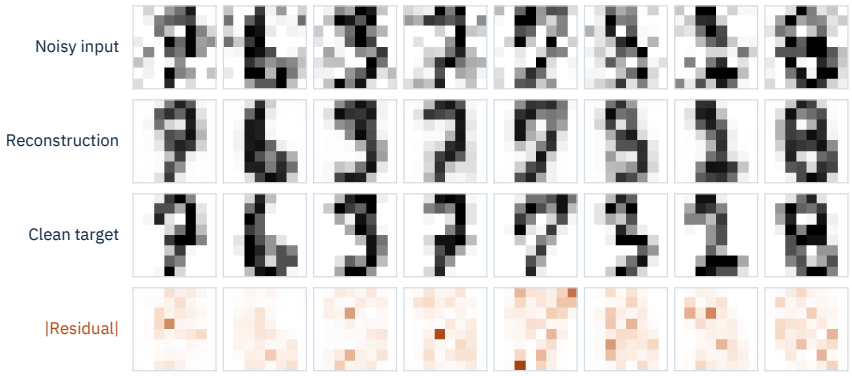
A powerful autoencoder may reconstruct anomalies too well. Reconstruction error is not automatically a calibrated anomaly probability; keep both caveats in mind for the next three slides.

Denoising trains from corrupted input to clean target

AUTOENCODERS AND 1-D SIGNALS

Denoising trains from corrupted input to clean target

Held-out error against the clean digits falls from 0.051 for the noisy input to 0.019.



Held-out MSE

vs clean target

noisy input 0.051

reconstruction 0.019

9,569 parameters

noise σ = 0.3, clipped

Small convolutional denoising autoencoder (9,569 parameters, seed 0, CPU) trained on scikit-learn digits; noise σ = 0.3, clipped.

Neural Networks / Day 4 · Convolutional neural networks 81

[3 min] A small denoising autoencoder trained for 100 epochs on 1,437 digits, each epoch with fresh Gaussian noise of standard deviation 0.3 (clipped to the 0–1 range). Rows: noisy input, reconstruction, clean target, and the absolute residual between reconstruction and target, all for held-out digits the model never saw.

On the 360 held-out digits, the mean squared error against the clean images is 0.051 for the noisy input and 0.019 for the reconstruction. The residual row shows where it still fails: isolated pixels at stroke ends and thin parts of digits. The first eight held-out digits are 7, 6, 3, 7, 7, 3, 2 and 8; compare the three 7s. A denoiser learns what typical digits look like, so it can also pull an unusual shape towards a typical one, a risk if the unusual detail was the point.

Lab 2A repeats this on MNIST at 28 × 28 with the architecture from the previous slide.

Reconstruction quality needs more than a low loss



[2 min] A low average loss can hide exactly the failure you care about. Compare noisy input, target, reconstruction and residual image, and look for structure in the residuals: a clean model leaves noise-like residuals, while systematic patterns mean the model is missing signal. Check whether small task-relevant defects disappear, because an image can look smoother while becoming less useful for an engineering decision.

Preserve consistent normalization between input and target, and keep the statistics with the model so deployment uses the same scaling.

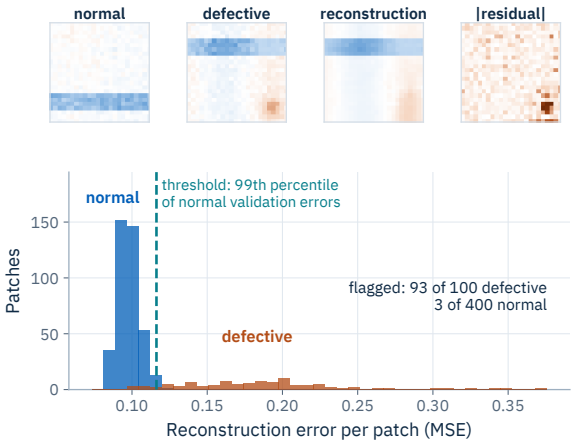
Match the output activation to the target range: a sigmoid output is appropriate only for a target constrained to 0 through 1, as in the Lab 2A notebook with MNIST. Standardized data such as thickness in standard units may require an unrestricted, linear output.

Reconstruction error can screen for anomalies

AUTOENCODERS AND 1-D SIGNALS

Reconstruction error can screen for anomalies

- Train on normal patches only; score each patch by its error.
- Set the threshold on normal validation data, never on test.
- The score is not a calibrated probability; strong models may rebuild defects too.



Synthetic casing patches (seed 11): 1,500 normal training, 400 normal validation, 400 normal and 100 defective test.

Neural Networks / Day 4 · Convolutional neural networks 83

[3 min] An autoencoder trained only on normal casing patches learns to reconstruct collars, azimuthal variation and noise; a pit it has never seen is reconstructed poorly, so its error is high. Top row: a defective patch, its reconstruction (the pit is largely missing) and the residual, which is bright exactly at the pit.

The threshold is the 99th percentile of errors on 400 normal validation patches. On the test set it flags 93 of 100 defective patches and 3 of 400 normal ones (0.75%). These are synthetic patches with a clean normal class; field data contain many more kinds of normal variation, so expect a much harder trade-off.

A powerful autoencoder may reconstruct anomalies too well. Reconstruction error is not automatically a calibrated anomaly probability. Select thresholds with independent validation and inspect how errors vary across normal operating regimes, such as different casing grades or tools. Module 5 of the Machine Learning course compares this with Isolation Forest and other detectors.

A 1-D convolution uses ordered neighbourhoods

AUTOENCODERS AND 1-D SIGNALS

A 1-D convolution uses ordered neighbourhoods

Quantity	PyTorch shape	Example
Input	B × C × L	Sensor channels along time or depth
Kernel	C_{out} × C × K	Local window over the ordered axis
Feature map	B × C_{out} × L_{out}	Learned local signal patterns

Order and sampling interval must have a real meaning.

[2 min] Conv1d is the same operation with one spatial axis. The input has shape batch × channels × length, where channels are different sensors or logs and length is time or depth. Each kernel spans all input channels and K consecutive samples; L_out follows the same output-size formula as before.

Do not apply 1-D convolution to arbitrary anonymous columns solely because they are numbered. For well logs, ensure regular or explicitly handled depth sampling and prevent windows crossing well boundaries. For forecasting, use causal receptive fields that exclude future inputs.

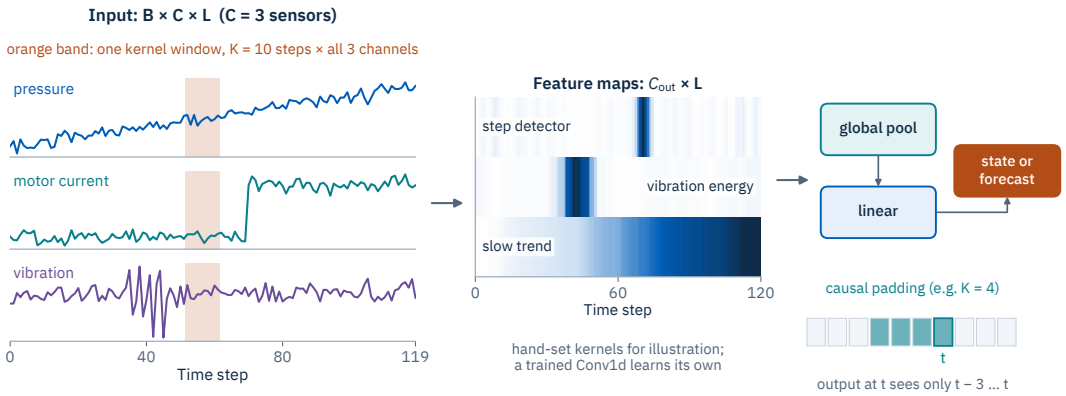
The receptive-field ledger from Part 3 converts directly into physical units here: a field of 18 samples at 0.1 m spacing is 1.8 m of log, at one-second sampling it is 18 seconds of sensor history.

1-D CNNs read multichannel sensor windows

AUTOENCODERS AND 1-D SIGNALS

1-D CNNs read multichannel sensor windows

Kernels slide along time across all channels; causal padding hides the future (Day 5).



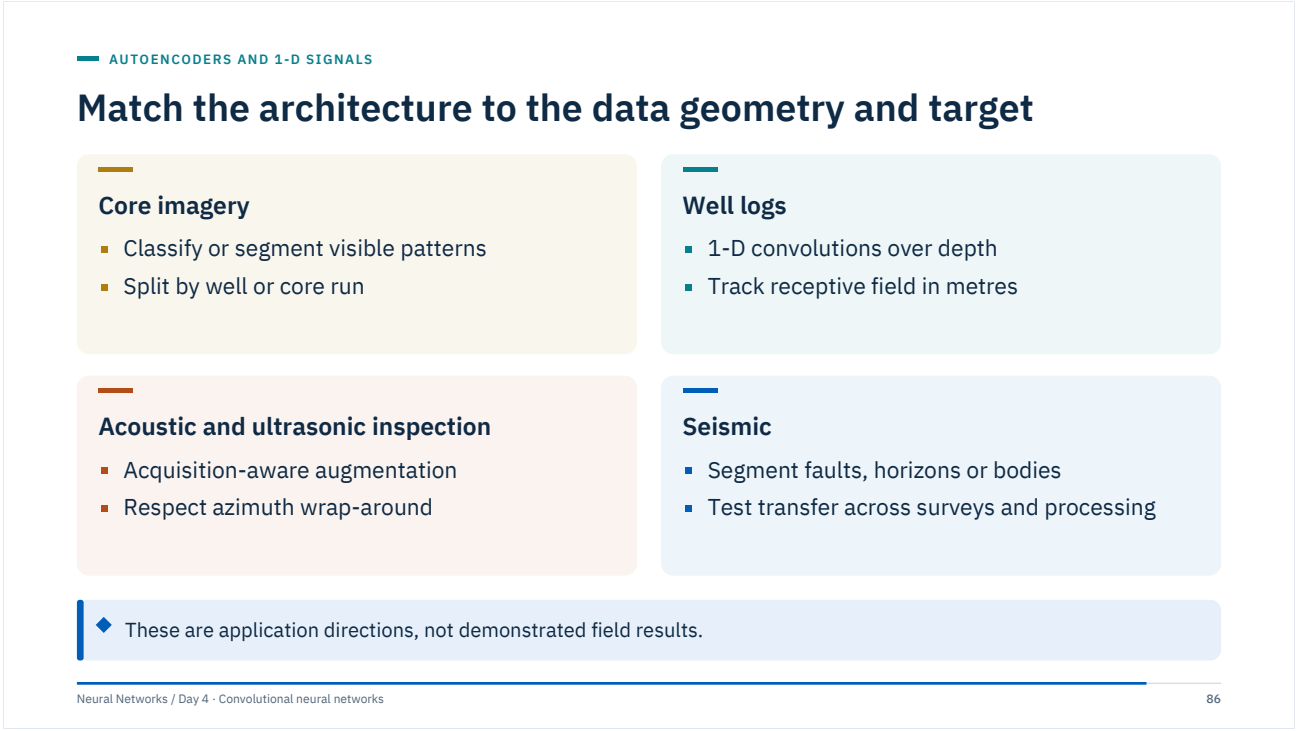
Synthetic three-sensor series (seed 21); hand-set kernels for illustration; schematic head and causal-padding inset.

Neural Networks / Day 4 · Convolutional neural networks 85

[3 min] Three synthetic sensor channels, for example pressure, motor current and vibration on a pump. A Conv1d kernel spans a window of K time steps across all three channels (the orange band) and slides along time, producing one feature map per filter. Three hand-set kernels illustrate the kind of patterns learned kernels can capture: a step detector responds where the current jumps, an energy filter where vibration bursts, a smoothing filter follows the slow pressure trend. A trained network learns its own kernels.

A global pooling layer and a linear head turn the maps into a state label or a forecast. For forecasting, padding must be causal: pad K – 1 steps on the left only, so the output at time t sees only t and earlier. Stacks of dilated causal convolutions are temporal convolutional networks, which Day 5 compares with recurrent networks and transformers, together with the time-ordered splits that sequence data require.

Match the architecture to the data geometry and target

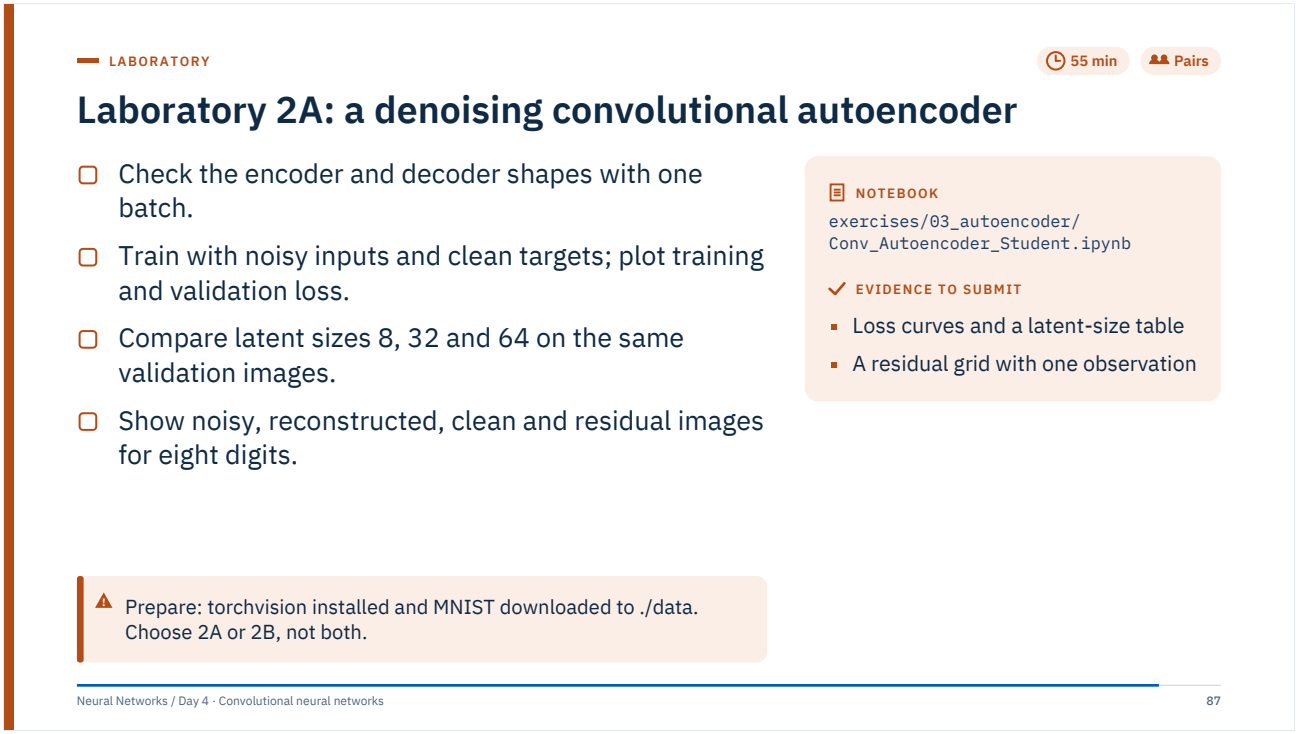


[3 min] Core imagery: classify or segment visible material patterns with source-level splits. Well logs: learn local depth patterns while tracking the receptive field in physical distance. Acoustic inspection: detect defects using acquisition-aware augmentation, including circular padding in azimuth. Seismic data: evaluate transfer across surveys and processing conditions.

These are application directions, not demonstrated field results. Connect every architecture choice to data geometry and the target. A classifier and a segmentation model produce different outputs and require different labelling and metrics.

Ask each table to pick one card and state, in one sentence, the output format, the split and one augmentation they would reject. Then introduce Lab 2.

Laboratory 2A: a denoising convolutional autoencoder



[55 min, 15:50–16:45] Suggested timing: 0–5 min shapes (1 × 28 × 28 → 64 × 1 × 1 → 1 × 28 × 28); 5–25 min training the latent-64 model; 25–45 min the latent-size comparison with fewer epochs each; 45–55 min residual grid and write-up.

Validation: the notebook holds out 5,000 training images and gives them one fixed, seeded noise draw, so every epoch and every latent size is scored on the same noisy images. It keeps the best validation epoch, compares latent sizes 8, 32 and 64 with the same 3-epoch budget, and uses the MNIST test set once, at the end (Section 8), next to the do-nothing reference: the error of the noisy input itself. The latent-64 model trains for 8 epochs; the solution runs in about 5–8 minutes on a laptop CPU and about 1 minute on Apple silicon or a GPU.

Expected pattern, to discuss rather than promise: smaller latent codes blur digits more; larger codes reconstruct more detail and more noise. Compare validation MSE and look at residuals.

Common errors: feeding the clean image as input (then there is nothing to denoise); a Sigmoid output with inputs normalized to mean 0; comparing models trained for different numbers of epochs. The notebook's t-SNE step on latent codes needs only scikit-learn.

Laboratory 2B: casing defects with grouped splits

LABORATORY

55 min · Pairs

Laboratory 2B: casing defects with grouped splits

- ☐ List runs, wells, missing values and flag fractions in the HDF5 file.
- ☐ Check the split by well (GroupShuffleSplit): wells, patches and prevalence per set.
- ☐ Train the CNN; choose the epoch and the threshold on validation wells.
- ☐ Report precision and recall per test well, once, at natural prevalence.
- ☐ Compare with a random patch split and explain the difference.

NOTEBOOK

`exercises/04_defect_detection/Defect_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Split table: wells and patches per set
- Per-test-well metrics; random versus grouped scores

⚠ Prepare: install h5py; the HDF5 file is in the exercise folder. Its labels are named "prediction": check their provenance.

Neural Networks / Day 4 · Convolutional neural networks 88

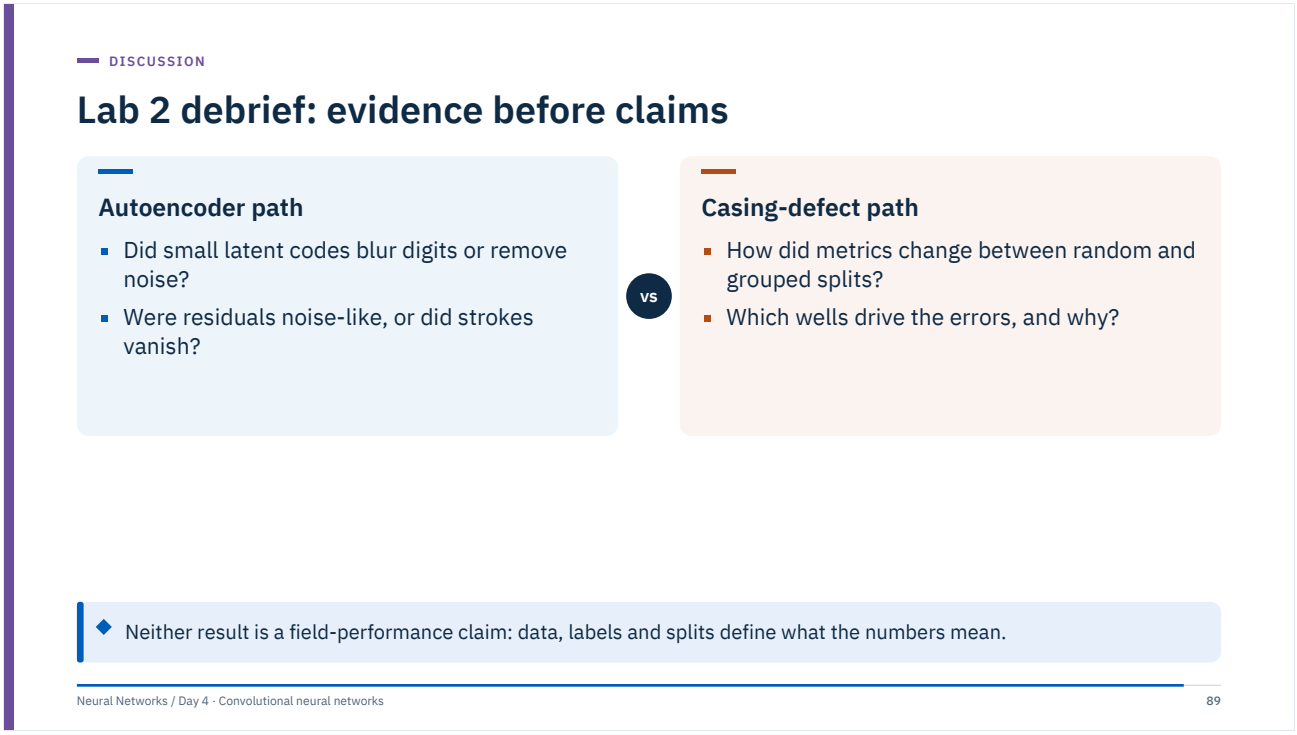
[55 min, 15:50–16:45] The notebook follows the grouped protocol from Part 5. The file holds 43 logging runs from 18 wells: each group name starts with the well name, and several wells have repeat passes over the same depths. THBK is a thickness deviation with no stored units; missing samples are coded −999.25, not NaN; one run (ConPr_R03_L002Up_USIT_15B) spans a range more than ten times wider than any other run and is excluded. Patches are 32 rows with stride 16, so neighbours overlap by half; every patch keeps its well.

Protocol: GroupShuffleSplit by well (seed 42) gives 11 training, 3 validation and 3 test wells, with patch prevalences of 36%, 6% and 22%. Rebalancing (at most 2 normal per defective patch) and the normalization statistics use training wells only. The epoch (best validation average precision) and the threshold (best validation F1) come from validation wells. Test wells are evaluated once, overall and per well, at natural prevalence, and the full-well plot uses a test well. Section 12 repeats the recipe with a random patch split of the development wells.

Labels: all_defects_prediction_V1 and V2 are the outputs of two versions of an earlier automated method, not verified ground truth, and the file has no attributes describing them. Of the pixels flagged by either version, only about 23% are flagged by both. A CNN trained on V1 learns to imitate V1: every score measures agreement with V1, and the write-up must say so.

Timing: 0–10 explore; 10–25 split and normalization; 25–45 train; 45–55 compare. The solution runs in about 5–8 minutes on a laptop CPU.

Lab 2 debrief: evidence before claims



[5 min, 16:45] Take one finding from each path.

Autoencoder path: ask for the latent-size table and one residual observation. Look for the trade-off between denoising and detail, and for the discipline of comparing on the same validation images with the same noise. In the solution run, after the same 3-epoch budget the validation MSE is about 0.039, 0.024 and 0.021 for latent sizes 8, 32 and 64 (the last digit varies between CPU and GPU runs), against about 0.12 for the noisy input itself. Ask learners to look in the latent-8 row for digits pulled towards a more typical shape of another class.

Casing-defect path: ask how metrics changed when the split moved from random patches to wells. A drop is the expected direction when overlapping patches no longer leak, but report what you observed rather than what you expected. In the solution run the average precision is 0.90 on randomly held-out patches, 0.69 on the validation wells and 0.30 on the test wells, against no-skill levels of 0.28, 0.06 and 0.22. Per-well results usually vary a lot; ask which wells drive the errors and whether they differ in tool, casing size or label density. Here most missed defects are in well 16_17-A16 (test recall 3%): its scores sit below the threshold chosen on the validation wells, and within the well the CNN ranks patches barely better than the thinnest-pixel baseline (AP 0.41 against 0.38, prevalence 0.28). In its long run, V1 flags scattered pixels that are barely thinner than their surroundings (median THBK −0.012, against −0.03 to −0.09 in most training runs).

Close with the callout. The labels in the casing file are themselves predictions of an earlier method, and the splits define what a score measures. Neither lab result is a field-performance claim.

Key takeaways

KEY TAKEAWAYS

Key takeaways

- ✓ Images are tensors whose axes have physical meaning; check shapes and spacing first.
- ✓ Convolution shares small local filters; padding, stride and dilation set the output size.
- ✓ Depth grows receptive fields; pooling and striding trade resolution for context.
- ✓ Residual paths, normalization and pretrained backbones make deep CNNs practical.
- ✓ Augmentations and splits must respect the physics and the data sources.
- ✓ Detection, segmentation and reconstruction change only the output.

Neural Networks / Day 4 · Convolutional neural networks 90

[2 min] Read the six statements and connect each to a slide or lab moment: the B × C × H × W code and the azimuth-wrap example; the output-size formula and parameter counts; the receptive-field ledger; the residual block, BatchNorm modes and transfer diagram; the label-changing transforms and the grouped split; and the four task formats, U-Net and autoencoders.

Ask each learner to pick the one statement they will apply first at work and name the dataset. That answer is often more useful feedback than the exit check itself.

Exit check: explain the mechanism, not the name

PRACTICE

5 min · Individual

Exit check: explain the mechanism, not the name

1. Why is a convolution's parameter count independent of image width?
2. How does a stride of 2 change spatial resolution and receptive field growth?
3. Why can frozen weights still coexist with changing BatchNorm behaviour?
4. Name one augmentation you rejected today, and why.
5. What would convince you that a defect model is not using a shortcut?

[5 min, written, individually] Answers:

1. The same kernel weights are reused at every position, so the count depends only on kernel size and channel counts, C_out × (C_in × K × K + 1), not on height or width.

2. It spaces output positions two input pixels apart, roughly halving height and width; later layers then add (K – 1) × jump to the receptive field with a doubled jump, so fields grow faster.

3. Running mean and variance are buffers, updated in training mode regardless of requires_grad; in evaluation mode the stored statistics are used. Frozen layers therefore need eval mode during head-only training.

4. Examples: 180° rotation of digits (6 becomes 9), polarity reversal when labels depend on reflection sign, horizontal flips of digits, random crops that can cut out the defect.

5. Consistent performance on held-out wells or surveys; counterfactual tests (remove artefacts, move or insert defects); evidence maps on the defects; no dependence on acquisition metadata.

References and further study

FURTHER STUDY

References and further study

- LeCun, Bottou, Bengio, Haffner (1998). Gradient-based learning applied to document recognition. *Proc. IEEE* 86(11).
- He, Zhang, Ren, Sun (2016). Deep residual learning for image recognition. *CVPR*. Ioffe, Szegedy (2015). Batch normalization. *ICML*.
- Ronneberger, Fischer, Brox (2015). U-Net: convolutional networks for biomedical image segmentation. *MICCAI*.
- Dumoulin, Visin (2016). A guide to convolution arithmetic for deep learning. arXiv:1603.07285. Zhang (2019). Making convolutional networks shift-invariant again. *ICML*.
- Geirhos et al. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence* 2. Goodfellow, Bengio, Courville (2016). *Deep Learning*, chapter 9.

Neural Networks / Day 4 · Convolutional neural networks 92

Primary sources for the day. LeCun et al. and He et al. anchor the architecture history; Ioffe and Szegedy introduced batch normalization; Ronneberger et al. introduced U-Net; Dumoulin and Visin is the clearest reference for padding, stride, dilation and transposed-convolution arithmetic; Zhang analyses why striding and pooling break shift equivariance; Geirhos et al. review shortcut learning. Goodfellow, Bengio and Courville's chapter 9 covers convolutional networks in depth. Other architectures on the timeline: Krizhevsky, Sutskever and Hinton (2012, AlexNet); Simonyan and Zisserman (2014, VGG); Szegedy et al. (2014, GoogLeNet); Huang et al. (2016, DenseNet); Howard et al. (2017, MobileNets); Tan and Le (2019, EfficientNet); Dosovitskiy et al. (2020, Vision Transformer); Liu et al. (2022, ConvNeXt).

The PyTorch documentation pages for Conv2d, BatchNorm2d, torchvision models and transforms, and the transfer-learning tutorial are linked in the sources; they are the authority for current API details such as the transforms v2 module and the weights enums. Point learners to the Day 5 references for temporal convolutional networks and sequence models.

Reference: https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html ; https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html ; https://docs.pytorch.org/vision/stable/models.html ; https://docs.pytorch.org/vision/stable/transforms.html ; https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html ; https://arxiv.org/abs/1603.07285